

QuantDinger — Technical Inventory

Comprehensive Module-by-Module Analysis for VIXOR Reuse
Decision

QuantDinger Technical Inventory

Source repo: `/home/z/my-project/audit/QuantDinger/` (commit observed: V3.1.0 "AI Agent Gateway / MCP HTTP / SSE `████`")

Audience: VIXOR build-vs-reuse decision

Author: Explore sub-agent

Date: 2026-05 (per CHANGELOG)

1. Executive Summary

QuantDinger is a **self-hosted, local-first AI quant operating system** that bundles, in a single Docker Compose stack, the full vertical slice of a retail/hobbyist algorithmic trading platform:

- Flask 3.1 backend (gunicorn, single-process + gthread, Postgres + Redis)
- Pre-built **Vue.js SPA** served by Nginx (Vue source lives in a private repo; this open-source tree only ships `frontend/dist/`)
- **Python strategy engine** with two authoring models:
 - `IndicatorStrategy` (dataframe-based, signal-style) — the primary model
 - `ScriptStrategy` (event-driven, `on_init(ctx) + on_bar(ctx, bar)` with `ctx.buy/sell/close_position`) — for live execution
- **Safe code execution** sandbox (`safe_exec.py`): regex + AST validation, restricted `__builtins__`, whitelisted imports (numpy/pandas/math/json/etc.), SIGALRM/ctypes-based timeout, optional multiprocessing subprocess isolation.
- **Backtest engine** (`backtest.py`, ~5K lines) with multi-timeframe (1m/5m) precision mode for crypto, long/short, leverage, commission/slippage, trailing stops, equity curve, trade ledger, persisted to Postgres.
- **Live trading** via direct REST clients (no ccxt for trade routing) for Binance (spot+futures), OKX, Bitget, Bybit, Coinbase Exchange, Kraken (spot+futures), KuCoin (spot+futures), Gate (spot+futures), Deepcoin, HTX, plus IBKR (US stocks via TWS/IB Gateway) and MT5 (Forex, Windows-only).
- **AI / Agent Gateway v1** (`/api/agent/v1`): capability-class scoped token auth (R/W/B/N/C/T), async job runner with SSE progress streaming, Idempotency-Key, audit log, paper-only trading by default, hard SaaS-mode guard that rejects T-scope tokens at issuance.
- **MCP server** (`mcp_server/`, also published to PyPI as `quantdinger-mcp`) exposing a curated read + backtest subset of the Agent Gateway as MCP tools for Cursor / Claude Code / Codex / OpenClaw / NanoBot. Supports stdio / sse / streamable-http transports.

- **Experiment orchestration** layer (`app/services/experiment/`): rule-based market regime detection, structured (grid/random) parameter tuning, and multi-round LLM-driven optimization with structured prompts.
- **Reflection / self-calibration loop**: a background worker validates past AI decisions against actual price outcomes, then re-calibrates BUY/SELL/HOLD score thresholds per market.
- **Notifications**: in-app + Email (SMTP) + Telegram (Bot API) + Twilio SMS
 - Discord webhook + generic webhook + browser push.
- **Multi-tenancy with billing**: credits, VIP plans, USDT-TRC20 on-chain payment (HD-derived per-order address, TronGrid reconciliation), OAuth (Google/GitHub), email verification, brute-force protection.
- **Strategy community marketplace**: indicator publish/purchase/comments/reviews with admin moderation.
- **Polymarket prediction-market module**: cache + AI divergence analysis + asset-level signal derivation.

Total scale: **~25 backend route files, ~40 service files, ~1,117 lines of SQL schema (30+ tables), ~10 live-trading adapters, 10 data sources, 9 data providers, 11 tests files, 1 MCP server (11 tools)**. Frontend ships as ~30 chunked JS bundles (~9MB+ minified) and supports 9 locales (zh-CN, zh-TW, en-US, ko-KR, ja-JP, fr-FR, de-DE, vi-VN, th-TH, ar-SA).

License: Apache-2.0 for code; trademarks/branding governed by a separate policy (no commercial use of the QuantDinger name/logo without written permission).

2. Tech Stack & Architecture

2.1 Stack matrix

Layer	Technology	Notes
Backend language	Python 3.10+ (Docker uses 3.12-slim-bookworm)	
Backend framework	Flask 3.1.3 + Werkzeug 3.1.6 + flask-cors 5.0.1	Single-process + gthread (gunicorn)
WSGI server	gunicorn 22+ (<code>gthread</code> worker, default 1 worker / 8 threads, 120s timeout)	<code>preload_app=False</code> so background threads survive fork
Database	PostgreSQL 16-alpine	Schema applied via <code>migrations/init.sql</code> mounted as <code>01-init.sql</code> in <code>docker-entrypoint-initdb.d</code>
DB driver	psycopg2-binary 2.9.9 with <code>ThreadedConnectionPool</code>	Pool size 5-50 (env: <code>DB_POOL_*</code>)

Layer	Technology	Notes
Cache	Redis 7-alpine (optional, falls back to in-memory <code>OrderedDict</code> LRU)	<code>maxmemory 128mb, allkeys-lru</code>
Auth	PyJWT 2.12+ HS256, 7-day expiry, <code>token_version</code> for single-client enforcement	OAuth2 (Google, GitHub) + email verification codes + brute-force throttling
Encryption	cryptography 43+ (Fernet, key derived via SHA-256(SECRET_KEY) → urlsafe-b64)	For <code>qd_exchange_credentials.encrypted_config</code>
Crypto data	CCXT 4.0+ (only used for <i>public</i> market data), direct REST for trading	
Stock data	yfinance, finnhub-python, akshare, Twelve Data, Tiingo, Tencent eastmoney/sina quotes	
Trading — crypto	Direct REST clients per exchange (no ccxt for orders)	11 exchanges
Trading — IBKR	ib_insync (TWS / IB Gateway)	US stocks only
Trading — MT5	MetaTrader5 (Windows-only)	Forex only
LLM providers	OpenRouter, OpenAI, Google Gemini, DeepSeek, Grok, MiniMax, Custom (OpenAI-compatible)	Auto-detect by configured API key; priority DeepSeek>Grok>MiniMax>OpenAI>Google>OpenRouter
Search APIs	Tavily (rotating keys), SerpAPI (rotating keys)	Optional, used for news enrichment
Frontend	Vue.js (Ant Design Pro fork — <code>vue-antd-pro</code> per <code>index.html</code> noscript tag), pre-built SPA	Source in private repo; this tree ships <code>dist/</code> only
Frontend server	Nginx 1.25-alpine (envsubst template → <code>default.conf</code>)	Proxies <code>/api/*</code> to backend; SPA fallback to <code>index.html</code>
MCP server	Python 3.10+, <code>mcp>=1.2.0</code> (FastMCP), httpx 0.27+	Published to PyPI as <code>quantdinger-mcp</code>
Container	Docker Compose v3.1+	4 services: postgres, redis, backend, frontend
Deployment targets	Local Docker, single-domain with host Nginx + Let's Encrypt, Railway (<code>railway.json</code> per service)	

2.2 Compose topology

```

Browser → (host Nginx 80/443) → frontend:80 (Nginx in container)
    |
    ├── serves dist/
    └── /api/* → backend:5000 (gunicorn)
                ├── postgres:5432 (pool 5–50)

```

- `BACKEND_URL` env var templated into `nginx.conf.template` via `entrypoint.sh` (custom envsubst on a strict shell-format list so nginx's own `$host` etc. stay literal).
- `BACKEND_PORT`, `DB_PORT` default to `127.0.0.1:5000/5432` so only 80/443 are public.
- `IMAGE_PREFIX` allows swapping Docker Hub for China mirrors (`docker.m.daocloud.io/library/`, `docker.xuanyuan.me/library/`).
- `ALLOW_LOCAL_DESKTOP_BROKERS=true` (default) gates IBKR/MT5 routes — set to `false` on SaaS cloud.

2.3 Process / thread model

- `create_app()` boots:
 - `init_database()` (verifies Postgres connection)
 - `ensure_admin_exists()` (creates admin user from `ADMIN_USER` / `ADMIN_PASSWORD`)
 - `register_routes(app)` — 22 blueprints + the `agent_v1` sub-package
 - Background threads (all daemon, all in-process, no Celery / no RQ):
 - `PendingOrderWorker` (polls `pending_orders` every 1s, batch 50)
 - `PortfolioMonitor` (scheduled AI analysis on manual positions)
 - `UsdtOrderWorker` (on-chain USDT payment reconciliation)
 - `PolymarketWorker` (caches prediction-market data)
 - `AICalibrationService.start_ai_calibration_worker` (offline threshold tuning)
 - `ReflectionService.start_reflection_worker` (daily validation cycle)
 - `restore_running_strategies()` (re-spawns `IndicatorStrategy` threads)
- `TradingExecutor` is a singleton; each running strategy owns its own thread.
- Unicorn: 1 worker × 8 threads (default). All background work runs inside that single worker process — multi-worker would duplicate background threads.

2.4 Two-topology design

A single binary serves both **self-hosted** (1 tenant, full feature set) and **SaaS/hosted** (N tenants, T-scope hard-blocked at issuance) modes. The selector is the env var `QUANTDINGER_DEPLOYMENT_MODE` (`saas/hosted/shared/multitenant/multi-tenant`). When in SaaS mode:

- `POST /api/agent/v1/admin/tokens` rejects any payload containing scope `T` with HTTP 403 (loud, not silent downgrade).
- `paper_only` is force-pinned to `True` on every issued token.
- `AGENT_LIVE_TRADING_ENABLED` is irrelevant because no token can have `T`.

Tested by `tests/test_agent_v1_saas_guard.py` (13 cases).

3. Backend Module Map

3.1 Routes — human-facing (mounted at `/api/...`)

File	Prefix	Routes (path · method · purpose)
health.py	(root)	GET / · GET /health · GET /api/health — liveness probe
auth.py	/api/auth	GET /security-config, POST /login, POST /login-code, POST /send-code, POST /register, POST /reset-password, POST /change-password, GET /oauth/google, GET /oauth/google/callback, GET /oauth/github, GET /oauth/github/callback, POST /logout, GET /info
user.py	/api/users	24 endpoints — list/export/detail/create/update/delete, reset-password, roles, set-credits, set-vip, credits-log, profile + update, my-credits-log, my-referrals, notification-settings GET/PUT + test, chart-templates GET/POST/DELETE, change-password, system-strategies, admin-orders, admin-ai-stats
kline.py	/api/indicator	GET /kline, GET /price
backtest.py	/api/indicator	GET /backtest/precision-info, POST /backtest, GET /backtest/history, GET /backtest/get, POST /backtest/aiAnalyze
market.py	/api/market	GET /config, GET /types, GET /menuFooterConfig, GET /symbols/search, GET /symbols/hot, GET /watchlist/get, POST /watchlist/add, POST /watchlist/remove, GET /watchlist/prices, GET /price, POST /stock/name
ai_chat.py	/api/ai	POST /chat/message, GET /chat/history, POST /chat/history/save

File	Prefix	Routes (path · method · purpose)
indicator.py	/api/indicator	GET /getIndicators, POST /saveIndicator, POST /deleteIndicator, GET /getIndicatorParams, POST /verifyCode, POST /aiGenerate, POST /codeQualityHints, POST /parseStrategyConfig, POST /callIndicator
strategy.py	/api	32 endpoints — templates, list/detail, backtest + history + get, create/batch-create/batch-start/batch-stop/batch-delete, update/delete, trades, positions, equityCurve, stop/start, test-connection, get-symbols, preview-compile, notifications (list/unread-count/read/read-all/clear), verify-code, ai-generate, performance, logs
credentials.py	/api/credentials	GET /desktop-brokers-policy, GET /list, GET /egress-ip, POST /create, DELETE /delete, GET /get
dashboard.py	/api/dashboard	GET /summary, GET /pendingOrders, DELETE /pendingOrders/<id>
settings.py	/api/settings	GET /schema, GET /public-config, GET /values, POST /save, GET /openrouter-balance, POST /test-connection
portfolio.py	/api/portfolio	positions CRUD, summary, monitors CRUD + run, alerts CRUD, groups (list + rename)
ibkr.py	/api/ibkr	GET /status, POST /connect, POST /disconnect, GET /account, GET /positions, GET /orders, POST /order, DELETE /order/<id>, GET /quote
mt5.py	/api/mt5	GET /status, POST /connect, POST /disconnect, GET /account, GET /positions, GET /orders, GET /symbols, POST /order, POST /close, DELETE /order/<ticket>, GET /quote

File	Prefix	Routes (path · method · purpose)
global_market.py	/api/global-market	GET /overview, GET /heatmap, GET /news, GET /calendar, GET /sentiment, GET /adanos-sentiment, GET /opportunities, POST /refresh
community.py	/api/community	indicators list/get/purchase/sync, my-purchases, comments (list/post/put/my), performance, admin pending-indicators/review-stats/review/unpublish/delete
fast_analysis.py	/api/fast-analysis	POST /analyze, POST /analyze-legacy, GET /history, GET /history/all, DELETE /history/<id>, POST /feedback, GET /performance, GET /similar-patterns
billing.py	/api/billing	GET /plans, POST /purchase, POST /usdt/create, GET /usdt/order/<id>
quick_trade.py	/api/quick-trade	POST /place-order, GET /balance, GET /position, POST /close-position, GET /history
polymarket.py	/api/polymarket	POST /analyze, GET /history
experiment.py	/api/experiment	POST /regime/detect, POST /pipeline/run, POST /ai-optimize (SSE), POST /ai-optimize-sync, POST /structured-tune, POST /save-strategy

3.2 Routes — Agent Gateway (mounted at /api/agent/v1)

File	Path · Method · Class	Purpose
health.py	GET /health (public), GET /whoami (R)	Liveness + token self-introspection
markets.py	GET /markets (R), GET /markets/<m>/symbols (R), GET /klines (R), GET /price (R)	Market data for agents
strategies.py	GET /strategies (R), GET /strategies/<id> (R), POST /strategies (W), PATCH /strategies/<id> (W)	Strategy CRUD; flipping status='running' requires T scope
backtests.py	POST /backtests (B)	Submit async backtest job (returns job_id)

File	Path · Method · Class	Purpose
experiments.py	POST /experiments/regime/detect (B, sync), POST /experiments/pipeline (B, async), POST /experiments/structured-tune (B, async), POST /experiments/ai-optimize (B, async, consumes LLM quota)	Experiment orchestration
portfolio.py	GET /portfolio/positions (R), GET /portfolio/paper-orders (R)	Manual positions + agent paper orders
quick_trade.py	POST /quick-trade/orders (T), POST /quick-trade/kill-switch (T)	Paper-only by default; live requires both <code>paper_only=false</code> on token AND <code>AGENT_LIVE_TRADING_ENABLED=true</code> env
jobs.py	GET /jobs (R), GET /jobs/<id> (R), GET /jobs/<id>/stream (R, SSE)	Async job polling + streaming
admin.py	POST /admin/tokens (admin JWT), GET /admin/tokens (admin JWT), DELETE /admin/tokens/<id> (admin JWT), GET /admin/audit (admin JWT)	Token lifecycle + audit viewer; SaaS-mode guard lives here
_helpers.py	—	<code>envelope()</code> / <code>error()</code> / <code>get_json_or_400()</code> / <code>clip_int()</code> shared by all agent routes

3.3 Services (app/services/)

Service	Responsibility
strategy.py (1374 LOC)	Strategy CRUD, list/get/create/update/delete, batch operations, <code>test_exchange_connection</code> , <code>get_exchange_symbols</code> , runtime metrics (realized/unrealized PnL), notification config helpers, bot-display builder (martingale/grid/trend/dca), JSON config parsing
strategy_compiler.py (689 LOC)	Compiles a declarative strategy spec JSON (entry rules, position config, pyramiding, risk management) into executable Python indicator code. Built-in indicator codegen for supertrend, EMA, RSI, MACD, Bollinger, KDJ, MA. Not the primary authoring path today (mostly superseded by IndicatorStrategy Python DSL).

Service	Responsibility
<code>strategy_script_runtime.py</code>	Runtime for <code>ScriptStrategy</code> — <code>ScriptBar</code> , <code>ScriptPosition</code> (long/short side-aware), <code>StrategyScriptContext</code> (<code>bars/param/log/buy/sell/closeposition</code>), <code>compile_strategy_script_handlers()</code> extracts <code>on_init/on_bar</code> from user code via <code>safeexec</code>
<code>backtest.py</code> (4974 LOC)	The heart of the backtest engine. <code>BacktestService</code> supports indicator-driven backtests (<code>run</code>) and snapshot backtests (<code>run_strategy_snapshot</code>), multi-timeframe (1m/5m) precision mode for crypto, long/short/both trade directions, leverage, commission/slippage, fixed + trailing stop-loss/take-profit, liquidation detection, candle-path simulation (bullish dip-then-rally vs bearish rally-then-dip), equity-curve + trade-ledger persistence to <code>qd_backtest_runs / _trades / _equity_points</code> . <code>ENGINE_VERSION = 'strategy-backtest-v1'</code>
<code>trading_executor.py</code> (3849 LOC)	Singleton live-trading executor. Spawns one daemon thread per running strategy. Pulls K-lines/prices, computes signals, enqueues orders into <code>pending_orders</code> . Strict state machine (flat → long/short → close-only/reduce-only). Per-candle signal de-dup. Position-state machine (single-direction per symbol in local mode). Price cache (10s TTL). Resource monitoring via <code>psutil//proc/self/status</code>
<code>pending_order_worker.py</code> (2439 LOC)	Polls <code>pending_orders</code> every 1s, dispatches via <code>live_trading.factory.create_client</code> + <code>place_order_from_signal</code> . Stuck-order reclaim (90s). Position-sync self-check against exchange every 10s. Records trades via <code>live_trading.records</code> . IBKR/MT5 lazy-imported
<code>live_trading/factory.py</code>	<code>create_client(exchange_config, market_type)</code> — returns direct REST client for 11 exchanges (Binance, OKX, Bitget, Bybit, Coinbase, Kraken, KuCoin, Gate, Deepcoin, HTX) + IBKR + MT5. Honors demo/testnet flags. Also exposes <code>query_fee_rate()</code> and <code>exchange_demo_mode_enabled()</code>
<code>live_trading/base.py</code>	<code>BaseRestClient</code> — minimal <code>requests</code> -based REST wrapper with cross-platform SSL verify resolution (looks for OS CA bundles, <code>certifi</code> , custom <code>LIVE_TRADING_CA_BUNDLE</code>), <code>LiveOrderResult</code> dataclass, <code>LiveTradingError</code> exception
<code>live_trading/binance.py</code> , <code>binance_spot.py</code> , <code>okx.py</code> , <code>bitget.py</code> , <code>bitget_spot.py</code> , <code>bybit.py</code> , <code>coinbase_exchange.py</code> , <code>kraken.py</code> , <code>kraken_futures.py</code> , <code>kucoin.py</code> , <code>gate.py</code> , <code>deepcoin.py</code> , <code>htx.py</code>	Per-exchange direct REST clients — <code>place_order</code> , <code>cancel_order</code> , <code>get_balance/get_account</code> , <code>get_positions</code> , <code>get_fee_rate</code> , etc. Each handles its own signing (HMAC-SHA256 for most; passphrase for OKX/Bitget/KuCoin/Coinbase; broker codes for Bybit/OKX/Bitget/Gate/HTX)

Service	Responsibility
<code>live_trading/execution.py</code>	<code>place_order_from_signal()</code> — converts a <code>pending_orders</code> row + signal payload into a real <code>LiveOrderResult</code> via the appropriate client. Symbol normalization (e.g. <code>BNB/USDT</code> → <code>BNB/USDT:USDT</code> for OKX swap)
<code>live_trading/records.py</code>	<code>apply_fill_to_local_position()</code> + <code>record_trade()</code> — write-back of fills into <code>qd_strategy_positions</code> and <code>qd_strategy_trades</code>
<code>live_trading/symbols.py</code>	Per-exchange symbol munging (<code>to_okx_swap_inst_id</code> , <code>to_gate_currency_pair</code> , etc.)
<code>mt5_trading/client.py</code> (858 LOC)	<code>MT5Client</code> — wraps <code>MetaTrader5</code> Python lib. Market/order/limit/stop order types, position close, account info. Requires Windows + MT5 terminal
<code>mt5_trading/symbols.py</code>	Forex symbol normalization (<code>EURUSD</code> → <code>EURUSD.m</code> etc.)
<code>ibkr_trading/client.py</code> (555 LOC)	<code>IBKRClient</code> — wraps <code>ib_insync</code> . Stock/option order placement, position/account/portfolio queries. Requires TWS or IB Gateway running locally
<code>ibkr_trading/symbols.py</code>	IBKR contract normalization (smart routing, CONID/futures month formatting)
<code>ai_calibration.py</code>	Offline threshold tuning. Reads <code>qd_analysis_memory</code> rows where <code>actual_return_pct</code> is known, searches a candidate abs-threshold grid (<code>[10,12,14,16,18,20,22,25,30]</code>), scores BUY/SELL/HOLD accuracy, persists best threshold into <code>qd_ai_calibration</code> . Background worker (<code>start_ai_calibration_worker</code>) runs daily
<code>reflection.py</code>	<code>ReflectionService.run_verification_cycle()</code> — for every unvalidated AI analysis older than <code>REFLECTION_MIN_AGE_DAYS</code> (default 7), fetches current price, computes <code>actual_return_pct</code> , sets <code>was_correct</code> . If anything was validated, runs <code>AICalibrationService.calibrate_market()</code> . Background worker runs every 24h
<code>analysis_memory.py</code>	<code>AnalysisMemory</code> dataclass + <code>get_analysis_memory()</code> singleton. Validates past BUY/SELL/HOLD decisions against actual price movement. Powers "self-tuning" feedback loop

Service	Responsibility
<code>llm.py</code> (629 LOC)	LLMService — 7 providers (OpenRouter, OpenAI, Google Gemini, DeepSeek, Grok, MiniMax, Custom OpenAI-compatible). Auto-detect by API key. OpenAI-compatible path used by DeepSeek/Grok/OpenRouter. Gemini path is bespoke (generateContent endpoint). Fallback-model support. OpenRouter-specific diagnostics (403/404 hints). call_llm_api() is the primary entry
<code>signal_notifier.py</code> (912 LOC)	SignalNotifier — multi-channel signal notifications: browser (in-app), email (SMTP/TLS/SSL), Telegram Bot, Twilio SMS, Discord webhook, generic webhook. User-level timezone resolution. HMAC-signed webhook support
<code>email_service.py</code> (362 LOC)	EmailService — verification codes (6-digit, 10-min expiry, max-5 attempts / 30-min lockout), SMTP send with TLS/SSL toggle
<code>oauth_service.py</code> (715 LOC)	Google + GitHub OAuth2. CSRF state persisted in qd_oauth_states (cross-worker safe). Account linking + unlinking. Auto-create user on first OAuth login
<code>usdt_payment_service.py</code> (830 LOC)	USDT-TRC20 payment via HD-derived per-order addresses (bip-utils xpub), TronGrid on-chain reconciliation worker. Auto-confirms paid → confirmed after N confirmations
<code>billing_service.py</code> (758 LOC)	Credits accounting (qd_credits_log), VIP plan grant/refund, feature-cost config (BILLING_AI_ANALYSIS , etc.), mock + USDT payment paths
<code>fast_analysis.py</code> (2805 LOC)	FastAnalysisService 3.0 — single LLM call with strongly-constrained prompt → structured analysis (decision, consensus score, trend outlook 24h/3d/1w/1m, trading plan with SL/TP, geopolitical detection). Backed by MarketDataCollector . Stores results in qd_analysis_memory for reflection loop
<code>market_data_collector.py</code> (2217 LOC)	Unified data fetcher for AI analysis — price/K-line via DataSourceFactory , macro (VIX, DXY, TNX, Fear&Greed) via global_market , news via Finnhub, fundamentals via Finnhub/yfinance. Technical indicator computation (_calculate_indicators : Wilder RSI, EMA-MACD, MA5/10/20, Pivot, swing high/low, Bollinger 20/2, Wilder ATR, volume ratio, range position, composite support/resistance)
<code>indicator_params.py</code> (380 LOC)	StrategyConfigParser (parses # @strategy stopLossPct 0.03 style annotations with type validation), IndicatorParamsParser (parses # @param name type default description), IndicatorCaller (allows one indicator to invoke another)

Service	Responsibility
<code>indicator_code_quality.py</code> (206 LOC)	Heuristic static analyzer — checks for <code>my_indicator_name/description</code> , <code>df.copy()</code> , <code>output = {...}</code> , <code>df['buy']/df['sell']</code> , presence of SL/TP annotations, unknown <code>@strategy</code> keys. Returns severity-tagged hints
<code>builtin_indicators.py</code>	Seeds new-user accounts with sample indicators (RSI edge trigger, dual-MA cross, Bollinger squeeze, MACD divergence, etc.)
<code>kline.py</code>	<code>KlineService</code> — K-line fetch with cache (Redis or in-memory), timeframe normalization
<code>portfolio_monitor.py</code> (1770 LOC)	<code>PortfolioMonitorService</code> — runs scheduled AI analysis on manual positions, evaluates alerts (price above/below, PnL above/below), sends localized notifications
<code>polymarket_worker.py</code> / <code>polymarket_analyzer.py</code> / <code>polymarket_batch_analyzer.py</code>	Polymarket prediction-market crawler, AI analyzer (YES/NO divergence, opportunity score), batch analyzer for asset-level signals
<code>community_service.py</code>	Indicator marketplace — publish/purchase/sync/comments/reviews/admin moderation
<code>user_service.py</code>	User CRUD, <code>ensure_admin_exists</code> , role enforcement
<code>security_service.py</code>	Brute-force protection, login attempt tracking, IP/identifier throttling
<code>search.py</code>	Tavily/SerpAPI search wrapper with rotating keys
<code>strategy_snapshot.py</code>	Snapshot serialization for experiment pipeline
<code>symbol_name.py</code>	Symbol display-name resolution per market

3.4 Experiment sub-package (`app/services/experiment/`)

File	Responsibility
<code>regime.py</code>	<code>MarketRegimeService.detect(df, ...)</code> — rule-based regime classifier. Extracts 6 features (priceChangePct, emaGapPct, realizedVolPct, atrPct, directionalEfficiency, volumeRatio) and classifies into one of 5 regimes: <code>bull_trend</code> , <code>bear_trend</code> , <code>range_compression</code> , <code>high_volatility</code> , <code>transition</code> . Each regime carries recommended strategy families (e.g. <code>trendfollowing</code> , <code>meanreversion</code> , <code>volatility_breakout</code>). Also builds segment-level regime timeline (<code>_build_segments</code>)

File	Responsibility
evolution.py	StrategyEvolutionService.build_variants(base_snapshot, parameter_space, max_variants, method) — generates strategy variants via grid (itertools.product) or random sampling over a structured parameter space. Supports list-of-values and {min,max,step} range specs. Path normalization (strategyConfig.risk.stopLossPct → strategy_config.risk.stopLossPct)
scoring.py	StrategyScoringService.score_result(result, regime) — multi-factor score (return, annualreturn, sharpe, profitfactor, win_rate, drawdown, stability) with weighted sum (0.88) + regime-fit (0.12). Sample-size penalty if trades < 5 / 12. Letter grade A/B/C/D/E
runner.py	ExperimentRunnerService — orchestrates the 3 pipelines: run_pipeline (legacy grid), run_structured_tune (no LLM), run_ai_pipeline (multi-round LLM optimization with early-stop at score 82+). SSE-friendly via on_progress callback. save_as_strategy persists the winner as a qd_strategies_trading row
prompts.py	LLM prompt construction (SYSTEM_PROMPT + _ROUND_TEMPLATE), indicator-param extraction, candidate parsing from LLM JSON response

3.5 Data sources (app/data_sources/)

File	Class	Backends
base.py	BaseDataSource (ABC)	Defines get_kline(symbol, timeframe, limit, before_time, after_time) + get_ticker(symbol). Provides filter_and_limit, calculate_time_range, log_result (delay-detection) helpers. TIMEFRAME_SECONDS map
factory.py	DataSourceFactory	Singleton-per-market cache. normalize_market() handles aliases (crypto→Crypto, fx→Forex, rustocks→MOEX, etc.). get_source(market), get_kline(...), get_ticker(...)
crypto.py	CryptoDataSource	CCXT (default Coinbase, env-configurable). Symbol normalization (BTCUSDT → BTC/USDT, BTC/USDT:USDT → BTC/USDT). Markets lazy-loaded

File	Class	Backends
<code>cn_stock.py</code>	<code>CNStockDataSource</code>	Tiered fallback: Twelve Data (paid) → Tencent (daily/weekly) → yfinance → AkShare
<code>hk_stock.py</code>	<code>HKStockDataSource</code>	Tiered fallback: Twelve Data → Tencent → yfinance → AkShare
<code>us_stock.py</code>	<code>USStockDataSource</code>	yfinance (primary) + finnhub (real-time quotes)
<code>forex.py</code>	<code>ForexDataSource</code>	Twelve Data (primary) + yfinance fallback
<code>futures.py</code>	<code>FuturesDataSource</code>	yfinance (CL, GC, SI, NG, HG, ZC, ZS, ZW, ES, NQ)
<code>moex.py</code>	<code>MOEXDataSource</code>	Moscow Exchange ISS API (SBER, GAZP, etc.)
<code>tencent.py</code>	Tencent quote/kline helpers	eastmoney + sina + tencent quote sources for CN/HK stocks
<code>asia_stock_kline.py</code>	Multi-source k-line fetchers	Twelve Data / yfinance / AkShare adapters
<code>cn_hk_fundamentals.py</code>	CN/HK fundamentals	AkShare-based
<code>polymarket.py</code> (1225 LOC)	<code>PolymarketDataSource</code>	Gamma API (markets/events) + Data API (positions) + CLOB API (orderbook). 5-min DB cache
<code>rate_limiter.py</code>	<code>RateLimiter</code> , <code>retry_with_backoff</code> decorator, random-User-Agent pool	Anti-ban tooling for eastmoney/tencent/akshare
<code>circuit_breaker.py</code>	<code>CircuitBreaker</code> (CLOSED/OPEN/HALF_OPEN state machine)	Per-source failure tracking with cooldown (default 5 min realtime / 3 min akshare)
<code>cache_manager.py</code>	<code>DataCache</code> (LRU + TTL, thread-safe <code>OrderedDict</code>)	Generic K-line/ticker cache

3.6 Data providers (`app/data_providers/`)

Global market dashboard aggregators (all use a shared cache via `app.utils.cache.CacheManager`):

File	Responsibility
<code>crypto.py</code>	Crypto heatmap (top-N by volume/movers)
<code>forex.py</code>	Forex pairs (yfinance + Twelve Data)
<code>indices.py</code>	World stock indices (yfinance)
<code>commodities.py</code>	Commodities (yfinance: CL, GC, SI, NG, HG, ZC, ZS, ZW)
<code>news.py</code>	News feed (Finnhub + RSS)

File	Responsibility
heatmap.py	Cross-market heatmap aggregator
sentiment.py	Fear & Greed Index, alternative.me sentiment
adanos_sentiment.py	Adanos Market Sentiment (US stock tickers, optional API key)
opportunities.py	Trading opportunities scanner (unusual moves, breakouts)

3.7 Utils (app/utils/)

File	Responsibility
safe_exec.py	The sandbox — see §4.3 for full analysis
auth.py	JWT generation/verification (HS256 , 7-day expiry, token_version for single-client enforcement), @login_required / @admin_required decorators, g.user_id / g.role propagation
agent_auth.py	Agent token pipeline — qd_agent_* prefix, SHA-256 hash at rest, scope parsing (R/W/B/N/C/T), market/instrument allowlists, per-token sliding-window rate limiter (in-process), redacted audit logger (qd_agent_audit), with_idempotency(kind) context manager
agent_jobs.py	In-process ThreadPoolExecutor (default 4 workers, env AGENT_JOBS_MAX_WORKERS). Per-job deque(maxlen=200) progress ring. SSE-friendly stream_progress(job_id, since_seq) generator with idle timeout. Tenant-scoped get_job/list_jobs
cache.py	CacheManager singleton — Redis when available, thread-safe in-memory OrderedDict fallback. Per-key TTL
config_loader.py	load_addon_config() — reads operator-editable config (persisted in DB or env) for API keys, billing, rate limits, etc.
credential_crypto.py	Fernet encryption of qd_exchange_credentials.encrypted_config — key derived as urlsafe_b64encode(sha256(SECRET_KEY)) . Rotating SECRET_KEY invalidates all stored exchange credentials
db.py	Thin re-export of db_postgres.get_pg_connection
db_postgres.py	ThreadedConnectionPool (psycopg2, pool 5–50). RealDictCursor for dict-style rows. Health-check ping on acquire. ? placeholder → %s auto-conversion for legacy code paths
http.py	Shared requests.Session factory with retry/proxy

File	Responsibility
language.py	Accept-Language negotiation (en-US, zh-CN, zh-TW, ko-KR, ja-JP, fr-FR, de-DE, vi-VN, th-TH, ar-SA)
local_brokers.py	local_desktop_brokers_allowed() — gates IBKR/MT5 routes based on ALLOW_LOCAL_DESKTOP_BROKERS env (default true for self-host, false for SaaS)
logger.py	setup_logger / get_logger — rotating file + console handler
strategy_runtime_logs.py	append_strategy_log(strategy_id, level, message) — writes to qd_strategy_logs for dashboard display
db_postgres.py	Connection pool with auto-reconnect, ? → %s placeholder shim

3.8 Scripts (backend_api_python/scripts/)

Script	Purpose
run_calibration.py	Cron-friendly wrapper: AICalibrationService.calibrate_market() per market
run_reflection_task.py	Cron-friendly wrapper: ReflectionService.run_verification_cycle()
backfill_zero_trades.py	Backfill zero-trade backtest records
simulate_trading_executor.py	Mock trading executor for integration testing
verify_moex.py	Live smoke-test of MOEX ISS API (python scripts/verify_moex.py SBER 1D 30)

4. Strategy Engine Deep Dive

This is the most valuable part of QuantDinger for VIXOR. The system supports **two distinct authoring models** that share storage but execute differently.

4.1 Authoring model A — IndicatorStrategy (primary)

- **Language:** pure Python (pandas + numpy).
- **Convention:** user code receives **df** (OHLCV DataFrame) and must:
 1. Compute boolean **df['buy']** and **df['sell']** columns (edge-triggered recommended — **raw & ~raw.shift(1)**).
 2. Build an **output** dict with **name**, **plots** (overlay/non-overlay series), **signals** (buy/sell marks with text + color).
 3. Optionally declare

metadata via comments:

```
# @strategy stopLossPct 0.03
# @strategy takeProfitPct 0.06
# @strategy entryPct 1
# @strategy tradeDirection long
# @param rsi_len int 14 RSI period
```

- **@strategy keys** (parsed by `StrategyConfigParser`): `stopLossPct`, `takeProfitPct`, `entryPct`, `trailingEnabled`, `trailingStopPct`, `trailingActivationPct`, `tradeDirection` (long|short|both).
- **@param keys** (parsed by `IndicatorParamsParser`): allow runtime parameterization (int/float/bool/str) — surfaced in the IDE for sliders.
- **Backtest path:** `BacktestService.run(indicator_code, market, symbol, ...)` → `safe_exec_with_validation(code)` over an `exec_env` containing `np/pd/df` → extract `df['buy']/df['sell']` → simulate bar-by-bar with optional multi-timeframe precision (1m exec under 1D signal for crypto, ≤15 days; 5m exec for ≤1 year).
- **Live path:** `TradingExecutor.start_strategy(id)` spawns a daemon thread that polls K-line every `decide_interval` seconds, re-runs the indicator code, derives latest-bar signals, enqueues `pending_orders` rows.

4.2 Authoring model B — ScriptStrategy (event-driven)

- **Language:** pure Python (numpy + pandas).
- **Convention:** user code defines `on_init(ctx)` (optional) and `on_bar(ctx, bar)` (required). The `ctx` (`StrategyScriptContext`) exposes: `ctx.position` — `ScriptPosition` with `side/size/entry_price/ direction (+1/-1/0)`, `supports bool()/int()/float()` and `rich comparison operators`. `ctx.balance`, `ctx.equity`, `ctx.current_index` `ctx.bars(n=1)` — list of recent `ScriptBars` (`open/high/low/close/volume`) `ctx.param(name, default)` — typed parameter access `ctx.log(msg)` — append to in-process log buffer `ctx.buy(price=None, amount=None)` / `ctx.sell(...)` / `ctx.close_position()` — append to `_orders` queue, consumed by the runtime after each bar.
- **Compile:** `compile_strategy_script_handlers(code)` runs `safe_exec_with_validation` to extract the two callables.
- **Live path:** `TradingExecutor` advances `ctx.current_index` bar-by-bar in real time, calls `on_bar(ctx, bar)`, drains `_orders` into the `pending_orders` queue with `signal_type` derived from the order action and current position direction (e.g. `open_long` / `add_long` / `close_long` / `open_short` / `add_short` / `close_short`).
- **Backtest path:** `_run_script_strategy` in `BacktestService` mirrors this flow for historical simulation.

4.3 Safe execution sandbox (app/utils/safe_exec.py)

Three layers of defense:

1. **Static validation** (`validate_code_safety`):

- **Regex blacklist** for dangerous patterns: `os.system`, `subprocess`, `__import__`, `eval(/exec(/compile(/open(/getattr(/setattr(/__dict__/_class__/_subclasses__/_globals__/_code__`, `globals(/vars(/dir(/breakpoint(/`, plus `import os|sys|subprocess|shutil|signal|resource|pymysql|sqlite3|psycopg2|sqlalchemy|requests|urllib|http|socket|ftplib|telnetlib|smtplib|ssl|pickle|marshal|shelve|ctypes|cffi|multiprocessing|threading|concurrent|asyncio|importlib|imp|builtins|code|codeop|runpy|tempfile|glob|pathlib|io`. * **AST walk** (fail-closed if parse fails): reject `ast.Import` / `ast.ImportFrom` for any module root not in `SAFE_IMPORT_MODULES = {numpy, pandas, math, json, datetime, time, collections, functools, itertools, statistics, decimal, fractions, operator, copy}`. Reject `ast.Call` to dangerous call names (`eval/exec/compile/__import__/_getattr/_setattr/_delattr/_globals/_vars/_dir/_breakpoint/_open/_input/_exit/_quit`) and attribute access on dangerous modules. Reject `ast.Attribute` for dangerous dunder.

2. **Restricted `__builtins__`**: only a whitelisted set is available — types (`bool/int/float/str/list/dict/set/tuple/frozenset/...`), `math` (`abs/round/pow/min/max/sum`), iteration (`len/enumerate/zip/map/filter/sorted/reversed/iter/next/_all/_any`), strings (`repr/chr/ord/format/bin/hex/oct`), type checks (`isinstance/issubclass/hasattr/callable`), `print`, safe exceptions (`Exception/ValueError/TypeError/...`), `staticmethod/classmethod/property/super/object`, constants (`True/False/None/Ellipsis/NotImplemented`). `__import__` is replaced with `_make_safe_import()` that only allows whitelisted roots.

3. **Timeout + memory**:

- **Cross-platform timeout** via `timeout_context(seconds)`:
 - Unix main thread: `signal.SIGALRM`
 - Else (Windows or non-main thread): `threading.Timer` + `ctypes` `PyThreadState_SetAsyncExc` to inject `TimeoutError` into the target thread. * Optional `RLIMIT_AS` memory cap on Linux when `SAFE_EXEC_ENABLE_RLIMIT=true` (default false).

4. **Subprocess isolation** (`safe_exec_isolated`): optional mode that runs the user code in a `multiprocessing.Process` with its own memory space, `RLIMIT_AS` enforced, pickle-serialized result pipe. Used for the highest-isolation path (heavy indicator runs).

4.4 Backtest engine internals

- **Candle-path simulation**: for each bar, the engine infers a 4-point price path — bullish bar = `open` → `low` → `high` → `close` (dip then rally), bearish = `open` → `high` → `low` → `close` (rally then dip). This lets SL/TP trigger intra-bar with realistic ordering.

- **State machine**: `position` ∈ `{-size, 0, +size}` with `position_type` ∈ `{None, 'long', 'short'}`. Signals are queued, executed at next bar's open price with slippage applied (long buys above open, short sells below).

- **Risk features:** fixed SL/TP, trailing stop (activates after `trailingActivationPct`, trails by `trailingStopPct`), liquidation detection (capital < `min_capital_to_trade` → `is_liquidated=True`).
- **Trade-direction modes:** `long` (long-only), `short` (short-only), `both` (close-then-reverse on opposite signal).
- **Multi-timeframe (MTF):** when `enable_mtf=True` and market is crypto:
 - ≤15 days → execute on 1m bars (max ~21,600 candles)
 - 16–365 days → execute on 5m bars (max ~105,120 candles)
 - >1 year → MTF disabled, fall back to native timeframe
- **Persistence:** `qd_backtest_runs` (config snapshot, `codehash`, `resultjson`), `qd_backtest_trades` (per-trade record), `qd_backtest_equity_points` (equity curve samples). `run_type` ∈ {`indicator`, `strategy_indicator`, `strategy_script`}.
- **Storage schema auto-migration** on first use (`ensure_storage_schema()`)
— adds columns / creates tables if missing, so the system is resilient to partial schema states.

4.5 Live execution flow

```
TradingExecutor.start_strategy(id)
├─ load strategy row from qd_strategies_trading
├─ spawn daemon thread
├─ loop:
│   └─ fetch latest K-line (cached 10s)
│   └─ fetch current price (KlineService)
│   └─ run indicator code via safe_exec
│   └─ extract latest-bar signals (df['buy'] / df['sell'])
│   └─ dedup via (strategy_id, symbol, signal_type, signal_ts) → skip if seen in 2× timeframe window
│   └─ enforce state machine: flat → open_long/open_short; long → add_long/reduce_long/close_long; short
│   └─ compute stake amount from entry_pct × equity × leverage
│   └─ INSERT into pending_orders (status='pending')
│   └─ SignalNotifier.notify_signal() – fire Telegram/Email/etc.
│   └─ sleep decide_interval seconds

PendingOrderWorker (separate thread):
├─ poll pending_orders WHERE status='pending' every 1s
├─ claim batch (row-level lock via SELECT FOR UPDATE)
├─ create_client(exchange_config, market_type) via factory
├─ place_order_from_signal()
├─ on success: apply_fill_to_local_position(), record_trade(), status='executed'
├─ on failure: attempts++, last_error; if attempts >= max_attempts → status='failed'
└─ reclaim stuck orders older than 90s
```

4.6 Strategy templates (`app/data/strategy_templates.json`)

10 starter templates with bilingual names/descriptions, difficulty levels, default parameters, market applicability:

1. MA Crossover (trend, beginner)
2. RSI Oversold Bounce (mean_reversion, beginner)
3. Bollinger Band Squeeze Breakout (volatility, intermediate)

4. MACD Divergence (trend, intermediate)
5. Grid Trading (market_making, intermediate)
6. Dollar Cost Averaging (passive, beginner)
7. (plus 4 more — Supertrend, dual-timeframe, breakout, scalper per JSON)

4.7 AI strategy generation

- **POST /api/indicator/aiGenerate** and **POST /api/strategies/ai-generate** — ask the LLM (provider auto-detected) to produce indicator code from a natural-language description, with a strongly-constrained prompt that requires the output to follow the IndicatorStrategy convention (`my_indicator_name`, `my_indicator_description`, `df['buy']`, `df['sell']`, `output = {...}`).
- **POST /api/experiment/ai-optimize** — multi-round LLM-driven optimization that: 1. Detects current market regime via **MarketRegimeService**. 2. Builds a prompt with the indicator code, `@param` declarations, regime summary, and previous-round results. 3. Asks LLM for N candidate parameter sets (5 by default), each with `indicatorParams` + `riskParams` + `reasoning`. 4. Backtests each candidate, scores via **StrategyScoringService**. 5. Repeats up to `maxRounds` (default 3) or early-stops at score ≥ 82 . 6. Returns ranked candidates + best output, ready to save as a strategy.

5. Agent v1 System Deep Dive

5.1 Identity model

- **Token format:** `qd_agent_<urlsafe_32_bytes>` (prefix `qd_agent_`).
 - **Storage:** SHA-256 hash of full token stored in `qd_agent_tokens.token_hash`. **Full token shown once at issuance** — never retrievable later. Prefix (`qd_agent_xxxxxxx`) shown in admin UI / audit logs for traceability.
 - **Per-token attributes:**
 - `scopes` — CSV subset of {R, W, B, N, C, T} (default R)
 - `markets` — CSV allowlist or * (e.g. `Crypto,USStock`)
 - `instruments` — CSV allowlist or * (e.g. `BTC/USDT,ETH/USDT`)
 - `paper_only` — boolean (default `True`); T-scope tokens always start paper-only
- `rate_limit_per_min` — *sliding-window in-process limit* (default 60)
`status` — `active` / `revoked` / `expired` * `expires_at` — nullable timestamp

5.2 Auth pipeline (`agent_required(scope)` decorator)

1. `_ensure_schema()` — idempotent CREATE TABLE IF NOT EXISTS for `qd_agent_tokens` / `_jobs` / `_audit` / `_paper_order`
2. Extract Bearer token from Authorization header; reject 401 if missing/malformed
3. Lookup token by SHA-256 hash → reject 401 if not found
4. Verify status='active' → reject 401 if revoked

5. Verify expires_at not past → reject 401 if expired
6. Parse scopes; verify required scope is granted → reject 403 if missing
7. Check rate limit (in-process sliding window) → reject 429 if exceeded
8. Set g.agent_token + g.agent_user_id
9. Execute handler
10. _touch_token_last_used()
11. _audit(scope_class, status_code, payload_summary, duration_ms) – INSERT into qd_agent_audit with redacted

5.3 Idempotency

with_idempotency(kind) context manager:

- Reads Idempotency-Key header.
- Queries qd_agent_jobs for (agent_token_id, kind, idempotency_key).
- If found → yields existing job → route returns duplicate: true with original job_id.
- If not found → yields None → caller performs work, persists new job row.
- Enforced by a unique partial index on (agent_token_id, kind, idempotency_key) WHERE idempotency_key IS NOT NULL.

5.4 Async job runner (agent_jobs.py)

- **Why not Celery?** Local-first deployments don't want a broker. A bounded ThreadPoolExecutor keeps the operational surface small. The submit/poll contract is unchanged if you later swap to Celery/RQ.

- `**`submitjob(userid, agenttokenid, kind, request_payload, runner, idempotencykey=None)**: * INSERT INTO qdagentjobs (status='queued') → returns jobid (uuid4 hex). * Detects runner signature via inspect.signature: * runner(payload) -> result – no streaming * runner(payload, onprogress) -> result – streaming * Dispatches run() on the pool: 1. status='running', started_at=NOW() 2. Synthetic phase=running progress event 3. Call runner; on success → status='succeeded', result=..., terminal progress event 4. On failure → status='failed', error=tb[-2000:]`, terminal event`

- **Progress streaming:**
 - Per-job deque(maxlen=200) ring buffer keyed by job_id
 - Monotonic seq per event
 - Latest snapshot also persisted into qd_agent_jobs.progress JSONB so cold reconnects can replay from DB * stream_progress(job_id, since_seq=0, idle_timeout_s=60) generator yields {seq, ts, data, terminal} records; cleans up state after terminal

- **SSE endpoint** (GET /jobs/<job_id>/stream):
 - Frames: snapshot (current row) → progress (per event) → ping (~15s keepalive) → result (terminal). Reconnect via ?since=<seq> or standard Last-Event-ID header. If job already terminated at connect time → emit snapshot + result immediately and close.

5.5 Trading safety (class T)

Three independent gates, all required for live execution:

1. Token must have scope **T**.
2. Token must have **paper_only=false** (operator flips explicitly).
3. Server env **AGENT_LIVE_TRADING_ENABLED=true** (deployment kill switch).

Until all three are met, **POST /api/agent/v1/quick-trade/orders** records a **paper order** into **qd_agent_paper_orders** using the latest K-line close as the simulated fill price — so AI workflows can exercise the full round-trip without touching exchange credentials.

POST /api/agent/v1/quick-trade/kill-switch cancels all of the calling tenant's open paper orders in one call.

5.6 Deployment-mode guard (**admin.py**)

When **QUANTDINGER_DEPLOYMENT_MODE** ∈ {**saas**, **shared**, **hosted**, **multitenant**, **multi-tenant**}:

- **POST /admin/tokens** rejects any payload with **T** in scopes → HTTP 403 with explicit message (no silent downgrade).
- **paper_only** force-pinned to **True** regardless of payload.
- Both guards run at issuance time, so a SaaS instance never has an at-rest token capable of routing real-money trades — even with **AGENT_LIVE_TRADING_ENABLED=true** misconfigured.

5.7 Audit log

qd_agent_audit columns: **user_id**, **agent_token_id**, **agent_name**, **route**, **method**, **scope_class**, **status_code**, **idempotency_key**, **request_summary** (redacted JSONB, ≤8KB), **response_summary** (redacted JSONB), **duration_ms**, **created_at**.

- Redaction: **password**, **secret**, **token**, **apikey**, **api_key**, **authorization** keys are recursively replaced with **<redacted>**. Long strings (>500 chars) truncated. Lists truncated to 20 items.
- Indexed on (**user_id**, **created_at** DESC), `(agenttokenid, created\_at DESC)`, (scope_class)`.

6. MCP Server (**mcp_server/**)

6.1 What it is

A **thin MCP (Model Context Protocol) wrapper** around the Agent Gateway. The REST API remains the source of truth; MCP only re-shapes a curated subset for clients that prefer the protocol (Cursor, Claude-style desktops, OpenClaw, NanoBot, cloud agents).

- **Package:** `quantdinger-mcp` on PyPI (install via `pipx`, `uvx`, or `pip`)
- **Source:** `mcp_server/src/quantdinger_mcp/server.py` (~306 LOC)
- **Dependencies:** `mcp>=1.2.0` (FastMCP), `httpx>=0.27.0`
- **Python:** 3.10+
- **License:** Apache-2.0

6.2 Configuration

Env-only (works in both desktop and cloud without CLI dance):

Variable	Required	Default	Purpose
<code>QUANTDINGER_BASE_URL</code>	yes	—	Backend URL, e.g. <code>http://localhost:8888</code>
<code>QUANTDINGER_AGENT_TOKEN</code>	yes	—	A <code>qd_agent_*</code> token issued via <code>/api/agent/v1/admin/tokens</code>
<code>QUANTDINGER_MCP_TRANSPORT</code>	no	<code>stdio</code>	<code>stdio</code> / <code>sse</code> / <code>streamable-http</code> (aliases: <code>http</code> , <code>streaming-http</code> , <code>STREAMABLE_HTTP</code>)
<code>QUANTDINGER_MCP_HOST</code>	no	<code>127.0.0.1</code>	Bind host for HTTP transports
<code>QUANTDINGER_MCP_PORT</code>	no	<code>8000</code>	Bind port for HTTP transports
<code>QUANTDINGER_TIMEOUT_S</code>	no	<code>60</code>	Upstream HTTP timeout

6.3 Exposed tools (11 total)

Read-class (R):

Tool	Wraps	Purpose
<code>whoami()</code>	<code>GET /api/agent/v1/whoami</code>	Inspect calling token
<code>list_markets()</code>	<code>GET /api/agent/v1/markets</code>	Markets token may query
<code>search_symbols(market, keyword, limit)</code>	<code>GET /api/agent/v1/markets/<market>/symbols</code>	Find symbols
<code>get_klines(market, symbol, timeframe, limit, before_time)</code>	<code>GET /api/agent/v1/klines</code>	OHLCV bars
<code>get_price(market, symbol)</code>	<code>GET /api/agent/v1/price</code>	Latest price

Tool	Wraps	Purpose
<code>list_strategies(limit)</code>	<code>GET /api/agent/v1/strategies</code>	Tenant's strategies
<code>get_strategy(strategy_id)</code>	<code>GET /api/agent/v1/strategies /<id></code>	One strategy
<code>get_job(job_id)</code>	<code>GET /api/agent/v1/jobs/<id></code>	Poll a job
<code>list_jobs(kind, limit)</code>	<code>GET /api/agent/v1/jobs</code>	List recent jobs

Backtest-class (B):

Tool	Wraps	Purpose
<code>submit_backtest(code, market, symbol, timeframe, start_date, end_date, ...)</code>	<code>POST /api/agent/v1/backtests</code>	Queue a backtest
<code>regime_detect(market, symbol, timeframe, start_date, end_date)</code>	<code>POST /api/agent/v1/experiments/regime/detect</code>	Synchronous regime detection
<code>submit_structured_tune(payload)</code>	<code>POST /api/agent/v1/experiments/structured-tune</code>	Queue grid/random tuning

Trading-class (T): intentionally NOT exposed via MCP. Use the REST `/api/agent/v1/quick-trade/...` endpoints directly if/when trading is explicitly enabled for an agent.

6.4 Tests

`mcp_server/tests/test_transport_resolution.py` (5 tests):

- Default transport is `stdio`.
- Aliases resolve correctly (`http/streaming-http/STREAMABLE_HTTP` → `streamable-http`).
- Unknown transport → `SystemExit(2)`.
- `_apply_http_settings_from_env` tolerates missing/odd `settings` shape (defensive against FastMCP version drift).

6.5 Railway deployment

`mcp_server/railway.json` + `mcp_server/Dockerfile` allow deploying the MCP server as a standalone service on Railway (separate from the backend).

7. Data Sources & Providers Map

7.1 Per-market data source

Market	Class	Primary	Fallbacks
Crypto	<code>CryptoDataSource</code>	CCXT (default Coinbase, env-configurable)	—
USStock	<code>USStockDataSource</code>	yfinance	finnhub (real-time quotes)
CNStock	<code>CNStockDataSource</code>	Twelve Data (paid)	Tencent → yfinance → AkShare
HKStock	<code>HKStockDataSource</code>	Twelve Data	Tencent → yfinance → AkShare
Forex	<code>ForexDataSource</code>	Twelve Data	yfinance
Futures	<code>FuturesDataSource</code>	yfinance (CL/GC/SI/NG/HG/ZC/ZS/ZW/ES/NQ)	—
MOEX	<code>MOEXDataSource</code>	Moscow Exchange ISS API	—
Polymarket	<code>PolymarketDataSource</code>	Gamma API + Data API + CLOB API	DB cache (5min TTL)

7.2 Resilience layer

- **CircuitBreaker** (`circuit_breaker.py`): CLOSED/OPEN/HALF_OPEN state machine. Per-source failure tracking with cooldown. Default thresholds: realtime (2 failures → 3min cooldown), akshare (3 failures → 5min cooldown). When OPEN, requests short-circuit to the next fallback source.

- **RateLimiter** (`rate_limiter.py`): per-source min-interval + jitter. Pre-instantiated limiters for eastmoney (2s+1-3s jitter), tencent (1s+0.5-1.5s jitter), akshare (2s+1.5-3.5s jitter). Plus a `retry_with_backoff` decorator with exponential backoff ($\text{base} \times \exp^{(\text{attempt}-1)}$), capped at `max_delay`, $\pm 20\%$ jitter).

- **Random User-Agent rotation** (12 desktop UAs: Chrome/Firefox/Safari/Edge on Windows/Mac/Linux).

- **DataCache** (`cache_manager.py`): thread-safe `OrderedDict` LRU with per-key TTL. Default `max_size=1000`, `default_ttl=600s`.

- **KlineService**: K-line fetch with Redis cache (or in-memory fallback), per-timeframe TTL (1m→5s, 3m→30s, 5m→60s, 15m→300s, ..., 1D→300s).

- **Proxy bypass for CN financial sources**: `run.py` injects `NO_PROXY` for `.eastmoney.com`, `.sina.com.cn`, `.10jqka.com.cn`, `.ssec.com.cn`, `.szse.cn`, `.hexun.com`, `.cninfo.com.cn`, `.gting.cn`, `.qq.com`, `.tencent.com`, `.mairui.club`, `.akshare.xyz`, `.baostock.com`, etc., so domestic CN data sources bypass overseas proxies.

7.3 Global market dashboard providers

Provider	TTL	Backends
<code>crypto.py</code> (heatmap)	300s	Binance API
<code>forex.py</code> (pairs)	120s	yfinance + Twelve Data
<code>indices.py</code> (world indices)	120s	yfinance
<code>commodities.py</code>	120s	yfinance
<code>news.py</code>	180s	Finnhub + RSS
<code>heatmap.py</code>	120s	aggregator
<code>sentiment.py</code>	21600s (6h)	alternative.me Fear & Greed
<code>adanos_sentiment.py</code>	—	Adanos API (optional key)
<code>opportunities.py</code>	3600s	computed from price scans

8. Broker Integrations Map

8.1 Crypto (direct REST, no ccxt for orders)

Exchange	Class	Spot	Swap	Demo/Testnet	Notes
Binance	<code>BinanceFutureClient</code> / <code>BinanceSpotClient</code>	✓	✓ (USDT-M)	<code>demo-fapi.binance.com</code> / <code>demo-api.binance.com</code>	Broker IDs <code>A2NAPZAC</code> (spot) / <code>HBpUbQjT</code> (futures)
OKX	<code>OkxClient</code>	✓	✓	simulated trading flag	Broker code <code>56fa80b0ce8cBCDE</code> , passphrase required
Bitget	<code>BitgetMixClient</code> / <code>BitgetSpotClient</code>	✓	✓	simulated trading flag	Channel API code <code>qvz9x</code> , passphrase required
Bybit	<code>BybitClient</code>	✓	✓ (linear)	<code>api-testnet.bybit.com</code>	Broker referer <code>Ri001020</code> , hedge-mode toggle
Coinbase Exchange	<code>CoinbaseExchangeClient</code>	✓	—	<code>api-public.sandbox.exchange.coinbase.com</code>	Passphrase required
Kraken	<code>KrakenClient</code> / <code>KrakenFuturesClient</code>	✓	✓	<code>demo-futures.kraken.com</code>	—

Exchange	Class	Spot	Swap	Demo/Testnet	Notes
KuCoin	<code>KucoinSpotClient / KucoinFuturesClient</code>	✓	✓	<code>openapi-sandbox.kucoin.com / api-sandbox-futures.kucoin.com</code>	Passphrase required
Gate	<code>GateSpotClient / GateUsdtFuturesClient</code>	✓	✓	<code>api-testnet.gateio.ws / fx-api-testnet.gateio.ws</code>	Channel ID dinger
Deepcoin	<code>DeepcoinClient</code>	—	✓	not configured	Passphrase required
HTX	<code>HtxClient</code>	✓	✓	not configured	Broker ID <code>AA7b890547</code> , separate spot/futures base URLs

Each client implements: `ping()`, `get_account()/get_balance()`, `get_positions()`, `place_order(...)`, `cancel_order(...)`, `get_fee_rate(symbol, market_type)`.

8.2 Traditional brokers

Broker	Class	Market	Required env
Interactive Brokers	<code>IBKRClient</code> (ib_insync)	US stocks	TWS or IB Gateway running on host (<code>ibkr_host</code> , <code>ibkr_port</code> , <code>ibkr_client_id</code> , <code>ibkr_account</code>)
MetaTrader 5	<code>MT5Client</code> (MetaTrader5 lib)	Forex	Windows + MT5 terminal running (<code>mt5_login</code> , <code>mt5_password</code> , <code>mt5_server</code> , <code>mt5_terminal_path</code>)

Both are **gated** by `local_desktop_brokers_allowed()` — set `ALLOW_LOCAL_DESKTOP_BROKERS=false` to disable on SaaS cloud.

8.3 Credential storage

- All exchange credentials stored in `qd_exchange_credentials.encrypted_config` (TEXT) — Fernet-encrypted with key derived from `urlsafe_b64encode(sha256(SECRET_KEY))`.
- Plaintext never persisted; only `api_key_hint` (first 4 + last 4 chars) stored for UI display.
- `decrypt_credential_blob(stored)` is the single decryption path — raises

`ValueError` on wrong key, so rotating `SECRET_KEY` invalidates all stored credentials (intentional).

9. Database Schema

PostgreSQL 16, schema applied via `migrations/init.sql` on first container boot. 30+ tables, all prefixed `qd_` (except `pending_orders`).

9.1 Users & Auth (Section 1)

Table	Purpose	Key columns	Indexes
<code>qd_users</code>	User accounts	<code>id</code> , <code>username</code> (unique), <code>password_hash</code> , <code>email</code> (unique), <code>role</code> (admin/manager/user/viewer), <code>credits</code> , <code>vip_expires_at</code> , <code>vip_plan</code> , <code>vip_is_lifetime</code> , <code>email_verified</code> , <code>referred_by</code> , <code>notification_settings</code> (JSON), <code>chart_templates</code> (JSON), <code>timezone</code> , <code>token_version</code> (single-client enforcement), <code>last_login_at</code>	<code>idx_users_referred_by</code>
<code>qd_credits_log</code>	Credits ledger	<code>user_id</code> , <code>action</code> (recharge/consume/refund/adminadjust/vipgrant), <code>amount</code> , <code>balance_after</code> , <code>feature</code> (aianalysis/strategyrun/backtest), <code>reference_id</code> , <code>operator_id</code>	<code>userid</code> , <code>action</code> , <code>createdat</code>
<code>qd_membership_orders</code>	Mock-paid membership orders	<code>user_id</code> , <code>plan</code> (monthly/yearly/lifetime), <code>price_usd</code> , <code>status</code> (paid/pending/failed/refunded), <code>paid_at</code>	<code>user_id</code>

Table	Purpose	Key columns	Indexes
qd_usdt_orders	USDT-TRC20 payment orders	user_id, plan, chain (TRC20), amount_usdt, address_index (HD), address (unique per chain), status (pending/paid/confirmed/expired/cancelled/failed), tx_hash, paid_at, confirmed_at, expires_at	unique(chain, address), user_id, status
qd_oauth_states	Cross-worker OAuth CSRF state	state (PK), provider, redirect, expires_at	expires_at
qd_verification_codes	Email verification codes	email, code, type (register/login/resetpassword/changeemail/change_password), expires_at, used_at, attempts, last_attempt_at	email, type, expires_at
qd_login_attempts	Brute-force tracking	identifier, identifier_type (ip/account), attempt_time, success, ip_address, user_agent	(identifier, identifier_type), attempttime
qd_oauth_links	Google/GitHub account links	user_id, provider, provider_user_id (unique per provider), provider_email, access_token, refresh_token	user_id, provider
qd_security_logs	Security audit	user_id, action (login/logout/register/resetpassword/oauthlogin), ip_address, user_agent, details (JSON)	userid, action, createdat

9.2 Trading (Sections 2-6)

Table	Purpose	Key columns	Indexes
qd_strategies_trading	Strategy records	user_id, strategy_name, strategy_type (IndicatorStrategy/ScriptStrategy), market_category, execution_mode (signal), status (running/stopped), symbol, timeframe, initial_capital, leverage, market_type (swap/spot), exchange_config (JSON), indicator_config (JSON), trading_config (JSON), ai_model_config (JSON), decide_interval, strategy_group_id, strategy_mode, strategy_code (for ScriptStrategy), last_rebalance_at (for cross-sectional)	userid, status, groupid
qd_strategy_positions	Live positions	strategy_id, symbol, side (long/short), size, entry_price, current_price, highest_price, lowest_price, unrealized_pnl, pnl_percent, equity, UNIQUE(strategy_id, symbol, side)	userid, strategyid
qd_strategy_trades	Trade ledger	strategy_id, symbol, type (openlong/closeshort/etc.), price, amount, value, commission, commission_ccy, profit	userid, strategyid, created_at

Table	Purpose	Key columns	Indexes
pending_orders	Order queue (consumed by PendingOrderWorker)	strategy_id, symbol, signal_type, signal_ts, market_type, order_type, amount, price, execution_mode, status (pending/processing/executed/failed/cancelled), priority, attempts, max_attempts, last_error, payload_json, exchange_id, exchange_order_id, exchange_response_json, filled, avg_price, executed_at, sent_at, processed_at	userid, status, strategyid
qd_strategy_notifications	In-app signal notifications	strategy_id, symbol, signal_type, channels, title, message, payload_json, is_read	userid, strategyid, is_read
qd_strategy_logs	Runtime logs (for dashboard)	strategy_id, level, message, timestamp	strategy_id, timestamp
qd_indicator_codes	User-saved indicator code	user_id, name, code, description, is_buy, end_time, publish_to_community, pricing_type (free/paid), price, is_encrypted, vip_free, purchase_count, avg_rating, rating_count, view_count, review_status (approved/pending/rejected), review_note, reviewed_at, reviewed_by, source_indicator_id (for sync from upstream publisher)	userid, reviewstatus, sourceindicatorid

9.3 Market data (Sections 7, 10, 11, 14, 15, 16)

Table	Purpose
qd_market_symbols	Seed catalog of hot symbols per market (USStock, Crypto, Forex, Futures, CNStock, HKStock) with display name, exchange, currency, ishot, sortorder. UNIQUE(market, symbol). Pre-seeded with ~80 hot symbols.
qd_watchlist	Per-user watchlist. UNIQUE(user_id, market, symbol).
qd_analysis_tasks	AI analysis task tracking
qd_backtest_runs	Backtest run records (userid, indicatorid, strategyid, runtype, market, symbol, timeframe, startdate, enddate, initialcapital, commission, slippage, leverage, tradedirection, strategyconfig, configsnapshot, engineversion, codehash, status, result_json)
qd_backtest_trades	Per-trade records (runid, tradeindex, tradetime, tradetype, side, price, amount, profit, balance, reason, payload_json)
qd_backtest_equity_points	Equity curve points (runid, pointindex, pointtime, pointvalue)
qd_exchange_credentials	Fernet-encrypted exchange credentials (userid, exchangeid, apikeyhint, encrypted_config)
qd_manual_positions	Manual portfolio positions (userid, market, symbol, side, quantity, entryprice, entrytime, notes, tags, groupname)
qd_position_alerts	Price/PnL alerts (userid, positionid, alerttype, threshold, notificationconfig, isactive, istriggered, lasttriggeredat, triggercount, repeatinterval)
qd_position_monitors	Scheduled AI monitors (userid, name, positionids, monitortype, config, notificationconfig, lastrunat, nextrunat)

9.4 AI / Reflection (Section 19.5)

Table	Purpose
qd_analysis_memory	Stores every AI analysis (userid, market, symbol, decision, confidence, priceatanalysis, summary, reasons JSONB, scores JSONB, indicatorssnapshot JSONB, rawresult JSONB, consensuscore, consensusabs, agreementratio, qualitymultiplier, validatedat, actualoutcome, actualreturnpct, wascorrect, userfeedback, feedbackat). Powers the self-calibration loop.

9.5 Community marketplace (Section 21)

Table	Purpose
qd_indicator_purchases	Indicator purchase records (<i>indicatorid</i> , <i>buyerid</i> , <i>sellerid</i> , <i>price</i>). <i>UNIQUE(indicatorid, buyer_id)</i> .
qd_indicator_comments	Comments/ratings (<i>indicatorid</i> , <i>userid</i> , rating 1-5, <i>content</i> , <i>parentid</i> for threaded replies, <i>isdeleted</i>)

9.6 Quick trades & Polymarket (Sections 22-23)

Table	Purpose
qd_quick_trades	Manual/discretionary trades from Quick Trade Panel (<i>userid</i> , <i>exchangeid</i> , <i>symbol</i> , <i>side</i> , <i>ordertype</i> , <i>amount</i> , <i>price</i> , <i>leverage</i> , <i>marketttype</i> , <i>tpprice</i> , <i>slprice</i> , <i>status</i> , <i>exchangeorderid</i> , <i>filledamount</i> , <i>avgfillprice</i> , <i>source</i> – <i>airadar/aianalysis/indicator/manual</i> , <i>rawresult</i> JSONB)
qd_polymarket_markets	Cached prediction markets (<i>marketid</i> unique, <i>question</i> , <i>category</i> , <i>currentprobability</i> , <i>volume24h</i> , <i>liquidity</i> , <i>enddateiso</i> , <i>status</i> , <i>outcometokens</i> JSONB, <i>slug</i>)
qd_polymarket_ai_analysis	AI analysis records (<i>marketid</i> , <i>aipredictedprobability</i> , <i>marketprobability</i> , <i>divergence</i> , <i>recommendation</i> YES/NO/HOLD, <i>confidencescore</i> , <i>opportunityscore</i> , <i>reasoning</i> , <i>keyfactors</i> JSONB, <i>relatedassets</i> TEXT[])
qd_polymarket_asset_opportunities	Derived asset-level signals (<i>marketid</i> , <i>assetsymbol</i> , <i>assetmarket</i> , <i>signal</i> BUY/SELL/HOLD, <i>confidence</i> , <i>entrysuggestion</i> JSONB)

9.7 Agent Gateway (Section 30)

Table	Purpose
qd_agent_tokens	Token registry (<i>userid</i> FK, <i>name</i> , <i>tokenprefix</i> , <i>tokenhash</i> UNIQUE, <i>scopes</i> CSV, <i>markets</i> CSV, <i>instruments</i> CSV, <i>paperonly</i> , <i>ratelimitpermin</i> , <i>status</i> , <i>expiresat</i> , <i>lastusedat</i>). Indexes: <i>tokenhash</i> unique, <i>userid</i> , <i>status</i> .

Table	Purpose
qd_agent_jobs	Async job records (BIGSERIAL, jobid <i>uuid hex UNIQUE</i> , userid FK, agenttokenid FK, kind — backtest/experimentpipeline/structuredtune/aioptimize, status — queued/running/succeeded/failed/cancelled, request JSONB, result JSONB, error TEXT, progress JSONB, idempotencykey, startedat, finishedat). Indexes: userid, status, kind, unique(agenttokenid, kind, idempotencykey) WHERE idempotency_key IS NOT NULL.
qd_agent_audit	Audit log (userid, agenttokenid, agentname, route, method, scopeclass R/W/B/N/C/T, statuscode, idempotencykey, requestsummary JSONB redacted, responsesummary JSONB redacted, durationms). Indexes: (userid, createdat DESC), (agenttokenid, createdat DESC), scopeclass.
qd_agent_paper_orders	Paper-trading ledger for T-scope tokens (orderid <i>UNIQUE</i> , userid FK, agenttokenid FK, market, symbol, side, ordertype, qty, limitprice, fillprice, fillvalue, status — filled/cancelled/rejected, note). Indexes: (userid, createdat DESC), agenttokenid.

9.8 Notable schema patterns

- **All tables use SERIAL or BIGSERIAL PRIMARY KEY** (no UUIDs in DB).
- **Foreign keys** use **ON DELETE CASCADE** for ownership chains (user → strategies → positions/trades).
- **Soft migrations via DO \$\$... END \$\$ blocks** that check `information_schema.columns` before **ALTER TABLE ADD COLUMN** — so the same `init.sql` is idempotent across upgrades.
- **No row-level security (RLS)** — tenant isolation enforced at the application layer via `user_id` filters in every query.
- **No stored procedures, triggers, or views** — all logic in Python.
- **JSONB** used liberally for flexible payloads
(`exchange_config`, `trading_config`, `indicator_config`, `notification_config`, `request`, `result`, `progress`, `request_summary`, `response_summary`, `reasons`, `scores`, `indicators_snapshot`, `raw_result`, `outcome_tokens`, `key_factors`, `entry_suggestion`, `chart_templates`, `notification_settings`).

10. AI / LLM Integration

10.1 Provider matrix

Provider	Base URL	Default model	Fallback model	Auth
OpenRouter	https://openrouter.ai/api/v1	openai/gpt-4o	openai/gpt-4o-mini	Bearer + HTTP-Referer + X-Title headers
OpenAI	https://api.openai.com/v1	gpt-4o	gpt-4o-mini	Bearer
Google Gemini	https://generativelanguage.googleapis.com/v1beta	gemini-1.5-flash	gemini-1.5-flash	API key in query string
DeepSeek	https://api.deepseek.com/v1	deepseek-chat	deepseek-chat	Bearer
Grok (xAI)	https://api.x.ai/v1	grok-beta	grok-beta	Bearer
MiniMax	https://api.minimax.io/v1	MiniMax-M2.7	MiniMax-M2.7-hig hspeed	Bearer
Custom	(user-configured)	(user-configured)	—	Bearer, OpenAI-compatible

10.2 Provider selection

1. If `LLM_PROVIDER` env (or `llm.provider` config) set → use that explicitly.
2. Otherwise auto-detect by API key presence, priority order:
`DeepSeek > Grok > MiniMax > OpenAI > Google > OpenRouter`.
3. If none configured → default to OpenRouter (will fail on first call with a clear error message).

10.3 Call paths

- **OpenAI-compatible** (OpenAI, DeepSeek, Grok, OpenRouter, MiniMax, Custom): standard `/chat/completions` POST with optional `response_format={"type": "json_object"}` for JSON mode.
- **Google Gemini**: bespoke `_call_google_gemini()` that converts OpenAI message format to Gemini `contents/parts` format, with `systemInstruction` for system messages and `responseMimeType: "application/json"` for JSON mode.

10.4 Error handling

- Non-2xx responses produce detailed error messages with provider-specific hints (e.g. OpenRouter 403 → "API key/endpoint not found"; 404 → "endpoint not found/endpoint").

- Empty content → `ValueError("Model X returned empty content")`.
- Missing `choices` → `ValueError("API response is missing 'choices'")`.

10.5 Model normalization

`_normalize_model_for_provider(model, provider):`

- If provider is OpenRouter → keep `vendor/model` format.
- Otherwise → strip prefix (`openai/gpt-4o` → `gpt-4o`).
- If prefix doesn't match the active provider → fall back to provider's default model (prevents sending `gpt-4o` to DeepSeek).

10.6 AI surfaces

Feature	Endpoint	Service	LLM usage
Fast Analysis (instant)	<code>POST /api/fast-analysis/analyze</code>	<code>FastAnalysisService</code>	Single call, structured JSON output (decision, scores, trend outlook, trading plan)
Fast Analysis (legacy)	<code>POST /api/fast-analysis/analyze-legacy</code>	Same	Same + <code>fast_analysis</code> envelope
AI Chat	<code>POST /api/ai/chat/message</code>	<code>LLMService</code> direct	Conversational
Indicator code generation	<code>POST /api/indicator/aiGenerate</code>	<code>LLMService</code>	Produces <code>IndicatorStrategy</code> Python code
Strategy code generation	<code>POST /api/strategies/ai-generate</code>	<code>LLMService</code>	Same
Backtest AI analysis	<code>POST /api/indicator/backtest/aiAnalyze</code>	<code>LLMService</code>	Summarizes backtest results
Experiment — AI optimize	<code>POST /api/experiment/ai-optimize (SSE)</code>	<code>ExperimentRunnerService.run_ai_pipeline</code>	Multi-round optimization (3 rounds × 5 candidates default)
Agent — AI optimize	<code>POST /api/agent/v1/experiments/ai-optimize (async job)</code>	Same	Same, with SSE progress streaming
Portfolio monitor	Background worker	<code>PortfolioMonitorService</code>	Periodic analysis on manual positions
Polymarket deep analysis	<code>POST /api/polymarket/analyze</code>	<code>PolymarketAnalyzer</code>	YES/NO divergence, opportunity score

10.7 Self-calibration loop

`qd_analysis_memory (validated rows)`
 ↓
 ▼

```

ReflectionService.run_verification_cycle() (daily, background thread)
| 1. Validate unvalidated rows older than 7 days
|   - Fetch current price for (market, symbol)
|   - Compute actual_return_pct = (current - price_at_analysis) / price_at_analysis × 100
|   - Set was_correct based on decision:
|     BUY correct if return > +2%
|     SELL correct if return < -2%
|     HOLD correct if |return| <= 5%
|   - Set validated_at = NOW()
|
| ▼
AICalibrationService.calibrate_market(market)
| 2. For each candidate abs_threshold in [10,12,14,16,18,20,22,25,30]:
|   - For each validated row, predict decision from consensus_score:
|     score >= +thr → BUY
|     score <= -thr → SELL
|     else → HOLD
|   - Compute accuracy = correct_predictions / total_predictions
| 3. Pick threshold with best accuracy (min 80 samples)
| 4. INSERT into qd_ai_calibration (market, buy_threshold, sell_threshold, ...)
|
| ▼
FastAnalysisService reads qd_ai_calibration on next call
→ uses tuned thresholds for BUY/SELL/HOLD decisions

```

11. Notifications (Telegram/Email/SMS)

11.1 Channels (per-strategy notification_config JSON)

```

{
  "channels": ["browser", "email", "phone", "telegram", "discord", "webhook"],
  "targets": {
    "email": "foo@example.com",
    "phone": "+15551234567",
    "telegram": "12345678 or @username",
    "discord": "https://discord.com/api/webhooks/...",
    "webhook": "https://example.com/webhook"
  }
}

```

11.2 Telegram

- Bot Token via **TELEGRAM_BOT_TOKEN** env (admin-set, system-wide).
- Per-user Telegram Chat ID stored in user **notification_settings** JSON or per-strategy **targets.telegram**.
 - Supports multiple recipients (comma-separated) and group/channel IDs (negative numbers).
 - HTTP API: <https://api.telegram.org/bot<token>/sendMessage>.
 - HTML-formatted messages with strategy name, symbol, signal type, price, direction, timestamp (in user's timezone).

11.3 Email

- SMTP via `SMTP_HOST`, `SMTP_PORT` (default 587), `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_FROM`, `SMTP_USE_TLS` (default true), `SMTP_USE_SSL` (for port 465).
- Two services:
 - `EmailService` — verification codes (6-digit, 10-min expiry, max-5 attempts / 30-min lockout) and password reset flows. * `SignalNotifier` — strategy signal notifications (HTML-formatted).
- All emails are admin-configured (system-wide SMTP); users can't change SMTP settings, only their target address.

11.4 SMS (Twilio)

- `TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`, `TWILIO_FROM_NUMBER` env vars.
- Per-user phone number in `targets.phone`.
- Used for critical signal alerts (configurable per strategy).

11.5 Discord & generic webhook

- Discord: posts to a Discord channel webhook URL.
- Generic webhook: HMAC-SHA256 signed POST to any URL.
- Both configurable per-strategy in `targets.discord` / `targets.webhook`.

11.6 Browser (in-app)

- Always available as a fallback channel.
- Persists notifications into `qd_strategy_notifications` table for the notification bell icon in the SPA.
 - `GET /api/strategies/notifications` (paginated list), `GET /api/strategies/notifications/unread-count`, `POST /api/strategies/notifications/read` (mark one read), `POST /api/strategies/notifications/read-all`, `DELETE /api/strategies/notifications/clear`.

12. Auth & Security

12.1 Auth pipelines (two parallel, never crossable)

Pipeline	Used by	Token format	Storage	Decorator
Human JWT	Browser SPA, OAuth callbacks	<code>jwt.encode(payload, SECRET_KEY, HS256)</code> , 7-day expiry	Not stored server-side; <code>token_version</code> in DB enforces single-client	<code>@login_required</code> , <code>@admin_required</code>
Agent token	External AI agents, MCP server	<code>qd_agent_<urlsafe_32_bytes></code>	SHA-256 hash in <code>qd_agent_tokens.token_hash</code>	<code>@agent_required(scope)</code>

A JWT is **never** accepted on `/api/agent/v1/*` and an agent token is **never** accepted on `/api/*`. No accidental cross-use.

12.2 JWT specifics

- Algorithm: HS256.
- Payload: `exp` (7 days), `iat`, `sub` (username), `user_id`, `role`, `token_version`.
- Single-client enforcement:** every login increments `qd_users.token_version`; JWT verification checks DB `token_version` matches the one in the token — mismatch → reject (kicks older sessions).
- Auto-regenerated `SECRET_KEY` if running in production with the default example value (defensive; `run.py main()`).

12.3 OAuth2

- Providers: Google, GitHub.
- CSRF state stored in `qd_oauth_states` (cross-worker safe — not in-memory).
- State TTL: 20 min (env: `OAUTH_STATE_TTL_MINUTES`).
- Auto-creates user on first OAuth login; links to existing user if email matches.
- Frontend redirect: PC hash mode (`/#/user/login`) or mobile history mode (preserves full path) — auto-detected by URL shape.

12.4 Email verification & brute-force protection

- 6-digit numeric code, 10-min expiry, max-5 attempts / 30-min lockout per email.
- `qd_login_attempts` tracks per-IP and per-account attempts.
- `qd_verification_codes.attempts` and `last_attempt_at` enforce code attempt limits.

12.5 Credential encryption

- `cryptography.fernet.Fernet` with `key = urlsafe_b64encode(hashlib.sha256(SECRET_KEY.encode()).digest())`.
- `encrypt_credential_blob(plaintext_json)` → Fernet-encrypted string for DB storage in `qd_exchange_credentials.encrypted_config`.
- `decrypt_credential_blob(stored)` → plaintext JSON; raises `ValueError` on wrong key (so rotating `SECRET_KEY` invalidates all stored credentials — must re-enter exchange API keys).
- API key hint (`api_key_hint` column) stores first 4 + last 4 chars for UI display.
- Secrets are masked in logs via `mask_secret()` (e.g. `ABCD...wxyz`).

12.6 Rate limiting

- **Per-token agent rate limit:** in-process sliding window, `rate_limit_per_min` per token (default 60). Returns HTTP 429 with `retriable: true` when exceeded.
- **Per-IP / per-account login throttling:** `qd_login_attempts` table.
- **Per-data-source rate limiters:** `RateLimiter` instances for eastmoney/tencent/akshare (min-interval + jitter).
- **Circuit breaker:** per-source failure tracking with cooldown (3min realtime, 5min akshare).
- **External API rate limits:** Finnhub (60/min), CCXT (built-in `enableRateLimit: true`).

12.7 Code execution sandbox

See §4.3 for the full `safe_exec` analysis. Key points:

- Three-layer defense: regex blacklist → AST walk → restricted `__builtins__`.
- Whitelisted imports only: `numpy`, `pandas`, `math`, `json`, `datetime`, `time`, `collections`, `functools`, `itertools`, `statistics`, `decimal`, `fractions`, `operator`, `copy`.
- Cross-platform timeout (SIGALRM on Unix main thread, `ctypes PyThreadState_SetAsyncExc` elsewhere).
- Optional subprocess isolation via `multiprocessing.Process` with `RLIMIT_AS`.
- Fail-closed: AST parse failure → reject.

12.8 Other security features

- **SafeJSONProvider** in `app/__init__.py`: sanitizes NaN/Infinity to `null` in all JSON responses (RFC 8259 compliance).
 - **CORS**: enabled globally via flask-cors (consider tightening for production).
 - **Security headers**: `X-Frame-Options: SAMEORIGIN`, `X-Content-Type-Options: nosniff`, `X-XSS-Protection: 1; mode=block` in nginx.
 - **Audit logs**: `qd_security_logs` (human auth events) + `qd_agent_audit` (every agent call).
 - **Egress IP detection** for Binance IP whitelist debugging (<https://ifconfig.me/ip>).
-

13. Frontend

13.1 What's in this repo

Only the **pre-built SPA** under `frontend/dist/`. The Vue source lives in a private repository (`QuantDinger-Vue-src`); the open-source workflow is:

```
private Vue repo —npm run build—> dist/ —rsync—> frontend/dist/ —Docker build—> nginx:1.
```

13.2 Stack identification

From `frontend/dist/index.html`:

- `<noscript>We're sorry but **vue-antd-pro** doesn't work properly without JavaScript enabled.</noscript>` — confirms Vue + Ant Design Pro.
- `chunk-vendors.429ef519.js` (modern) + `chunk-vendors-legacy.d5daba6c.js` (IE/legacy) — Vue CLI build with `@vue/cli-service` legacy build mode.
- `app.01d755db.js` (modern) + `app-legacy.5ad1f339.js` (legacy).
- Module/nomodule split for modern vs legacy browsers.
- 9 locale chunks: `lang-zh-CN`, `lang-zh-TW`, `lang-ko-KR`, `lang-ja-JP`, `lang-fr-FR`, `lang-de-DE`, `lang-vi-VN`, `lang-th-TH`, `lang-ar-SA`.

13.3 Page structure (inferred from CSS chunk names)

CSS chunks suggest these view modules:

- `app.css`, `chunk-vendors.css` — base
- `user.css` — user/auth pages
- `theme-colors.css` — theme tokens
- Numeric chunks (668, 374, 482, 456, 298, 228, 593, 737, 676, 727, 412,

334, 18, 196, 506, 509, 89, 254, 130, 159, 912, 529, 362, 45) — Vue lazy-loaded route chunks. Roughly 25-30 distinct route views.

13.4 Deployment

- `frontend/Dockerfile`: `nginx:1.25-alpine` base, copies `dist/` to `/usr/share/nginx/html/`, copies `nginx.conf.template` to `/templates/`, copies `entrypoint.sh`.
- `frontend/entrypoint.sh`:
 - Strips whitespace and angle brackets from `BACKEND_URL`.
 - Defaults bare hosts to `https://` (Railway public domains are HTTPS).
 - Defaults: `BACKEND_URL=http://backend:5000`, `PORT=80`.
 - Explicit `envsubst '${BACKEND_URL} ${PORT}'` (so nginx's `$host`, `$remote_addr` etc. stay literal). * Hands off to the official nginx image's entrypoint chain.
- `frontend/nginx.conf.template`:
 - Gzip on, immutable caching for hashed static assets (1y).
 - `location /api/` → `proxy_pass ${BACKEND_URL}/api/` with 10-min read/send timeout for long-running backtests, 10MB body limit. `location /` → *SPA fallback* (`try_files $uri $uri/ /index.html`). `location /health` → 200 OK (container healthcheck).
- `frontend/railway.json`: Railway deployment config.

13.5 Maps & assets

- `frontend/dist/maps/world-atlas.json` + `world.json` — for global market heatmap visualizations (likely using ECharts or D3 geo projections).
- Logo, slogo (small), avatar2 (default user avatar), background SVG.

14. Tests

Location: `backend_api_python/tests/` (11 test files)

File	Scope	Coverage
<code>conftest.py</code>	Shared pytest fixtures — sets <code>SECRET_KEY</code> , <code>ADMIN_USER</code> , <code>ADMIN_PASSWORD</code> , <code>CACHE_ENABLED=false</code> ; creates app via <code>create_app("testing")</code> + test client	All tests
<code>test_health.py</code>	<code>GET /api/health</code> returns 200	Health endpoint

File	Scope	Coverage
<code>test_json_encoder.py</code>	NaN/Inf → null in JSON responses (RFC 8259 compliance)	<code>SafeJSONProvider</code>
<code>test_indicator_code_quality.py</code>	Heuristics for indicator code quality (empty code, missing output, missing SL/TP, unknown @strategy keys)	<code>indicator_code_quality.analyze_indicator_code_quality</code> + <code>StrategyConfigParser</code>
<code>test_data_providers.py</code>	Cache layer helpers (<code>safe_float</code> , <code>get_cached</code> , <code>set_cached</code> , <code>clear_cache</code>)	<code>app.data_providers</code> cache
<code>test_three_minute_timeframe.py</code>	CCXTConfig timeframe mapping includes 3m	Data source config
<code>test_global_market_adanos.py</code>	Optional Adanos global-market endpoint (skipped if no API key)	<code>global_market</code> route
<code>test_moex_data_source.py</code>	MOEX data source — mocks HTTP layer, doesn't hit real ISS API	<code>MOEXDataSource</code>
<code>test_agent_v1.py</code>	Agent Gateway smoke tests: <code>/health</code> public, <code>/whoami</code> requires token, unknown/inactive/expired tokens rejected, scope enforcement returns 403 not 401	<code>agent_required</code> decorator + <code>agent_v1</code> routes
<code>test_agent_v1_saas_guard.py</code>	SaaS deployment guard: <code>QUANTDINGER_DEPLOYMENT_MODE=saas</code> rejects T-scope at issuance with 403, force-pins <code>paper_only=True</code> ; 13 cases covering env-var spellings + self-hosted regression	<code>agent_v1/admin.py._is_saas_mode()</code>
<code>test_agent_jobs_progress.py</code>	Async job runner — runner-signature detection (single-arg vs <code>on_progress</code>), progress events accumulate monotonically, terminal events stop stream + clean up state, idle timeout returns control without hanging	<code>agent_jobs</code> progress / streaming machinery
<code>test_experiment_services.py</code>	Experiment orchestration — regime detection, evolution variants, scoring	<code>experiment.regime</code> / <code>evolution</code> / <code>scoring</code>

MCP server tests (`mcp_server/tests/test_transport_resolution.py`, 5 tests): transport resolution (stdio default, alias mapping, unknown exits), HTTP settings shim tolerates FastMCP version drift.

Test observations

- **No end-to-end / integration tests** that exercise the full strategy → backtest → live-trade flow.

- **No load / performance tests.**
 - **No tests for the strategy runtime itself** (`trading_executor.py`, `backtest.py`) — these are the most complex modules.
 - **No tests for the safe_exec sandbox** — concerning given it's the primary security boundary.
 - Most tests stub DB / HTTP / external services via monkeypatch.
 - `pytest` is the runner; CI command: `pytest tests/ -v`.
-

15. Notable Patterns Worth Reusing

15.1 Capability-class scoped agent tokens

The R/W/B/N/C/T taxonomy with per-token market/instrument allowlists + `paper_only` flag is a clean, auditable model for giving AI agents least-privilege access to a trading platform. The dual-gate design (token `paper_only=false` AND server `AGENT_LIVE_TRADING_ENABLED=true`) is a solid defense-in-depth pattern. The SaaS-mode issuance-time guard (rejecting T-scope outright rather than silently downgrading) is particularly well thought out.

15.2 Idempotency-Key with unique partial index

The Postgres pattern:

```
CREATE UNIQUE INDEX idx_agent_jobs_idem
  ON qd_agent_jobs(agent_token_id, kind, idempotency_key)
  WHERE idempotency_key IS NOT NULL;
```

...is an elegant way to enforce idempotency at the DB layer without bloating the index with NULL idempotency keys.

15.3 SSE progress streaming with resume

The `snapshot` → `progress` → `ping` → `result` SSE frame protocol with `?since=<seq>` (or `Last-Event-ID`) resume is a clean pattern for long-running jobs. The per-job `deque(maxlen=200)` ring buffer + latest-snapshot-in-DB hybrid gives both low-latency live streaming and cold-reconnect replay.

15.4 Three-layer safe code execution

Regex blacklist → AST walk → restricted `__builtins__` with whitelisted imports, plus cross-platform timeout (SIGALRM + ctypes fallback) and optional subprocess isolation. The fail-closed approach (AST parse failure → reject) is the right default.

15.5 Multi-tier data source fallback with circuit breaker

`CircuitBreaker` (CLOSED/OPEN/HALF_OPEN) + per-source `RateLimiter` + `retry_with_backoff` decorator + `DataCache` LRU is a robust pattern for surviving flaky public financial APIs (eastmoney/sina/akshare/tencent).

15.6 Self-calibration feedback loop

The `qd_analysis_memory` → `reflection` → `ai_calibration` → `fast_analysis` loop (validate past decisions → recompute thresholds → feed back into next analysis) is a practical, low-cost approach to making AI trading decisions self-tuning without requiring model retraining.

15.7 Single-binary dual-topology deployment

Same codebase serves self-hosted (1 tenant, full features) and SaaS (N tenants, T-scope hard-blocked) via a single env var. The guard runs at issuance time so existing tokens survive a mode switch (no destructive migration), but new issuances immediately follow the new rules.

15.8 JSON sanitization at the framework layer

`SafeJSONProvider` that recursively replaces NaN/Infinity with `null` before serialization is a one-line fix that prevents every frontend consumer from breaking on `JSON.parse()` of invalid JSON tokens. Worth adopting in any Flask project that returns float-heavy responses.

15.9 Cross-worker OAuth state via DB

Storing OAuth CSRF state in `qd_oauth_states` (not in-memory) is the only correct way to do OAuth in a multi-worker / multi-replica deployment. The pattern generalizes to any stateful callback flow.

15.10 Candle-path simulation for backtests

The 4-point intra-bar path inference (bullish = open→low→high→close, bearish = open→high→low→close) is a simple but effective heuristic for realistic SL/TP triggering within a single bar — better than assuming worst-case or best-case fills.

16. Gaps / Limitations

16.1 No real-money agent trading implemented

`agent_v1/quick_trade.py` returns HTTP 501 ("Live agent trading is not implemented in this build") even when both `paper_only=false` and `AGENT_LIVE_TRADING_ENABLED=true`. The full T-scope live execution path is tracked under roadmap A6 and **not yet shipped**. The trading surface exists only via the human Quick Trade flow and the `TradingExecutor` + `PendingOrderWorker` for human-created strategies.

16.2 Single-process background workers

All background workers (portfolio monitor, pending order worker, USDT order worker, polymarket worker, AI calibration, reflection, strategy restore) run as daemon threads inside the single gunicorn worker. This:

- Limits horizontal scaling (can't run multiple backend replicas without duplicating workers).
- Means a worker crash takes down the whole backend.
- No job queue / broker (Celery, RQ, Dramatiq) — operator must add one to scale.

16.3 Frontend source not in this repo

The Vue source is in a private repo. Open-source contributors can only modify pre-built `dist/` (which is minified and chunk-hashed). Frontend changes require access to the private repo. This is a deliberate choice (see `CONTRIBUTING.md`) but limits community contribution to backend.

16.4 Test coverage gaps

- No tests for `safe_exec.py` (the security boundary).
- No tests for `backtest.py` (5K LOC, the core engine).
- No tests for `trading_executor.py` (3.8K LOC, the live engine).
- No tests for `strategy_compiler.py` (codegen).
- No tests for any of the live_trading exchange clients.
- No end-to-end / integration tests.
- No load / performance tests.

16.5 No walk-forward / OOS validation

Per `docs/AI_TRADING_SYSTEM_PLAN_CN.md`, walk-forward and out-of-sample scoring are deferred to "Phase 3" and not yet implemented. The experiment scoring (`StrategyScoringService`) only evaluates on the same backtest window — overfitting risk is real.

16.6 No paper-trading champion promotion path

"Phase 4" (paper trading → champion strategy promotion) is also deferred. The Agent Gateway's T-scope paper-order ledger is the closest thing, but there's no automated champion-selection flow yet.

16.7 In-process rate limiting (not distributed)

The agent-token rate limiter uses an in-process `dict[int, list[float]]` sliding window. This works for single-instance deployments but breaks down with multiple backend replicas (each replica has its own counter). For SaaS multi-replica, you'd need Redis-based rate limiting.

16.8 No structured logging

Logging uses Python's stdlib `logging` with rotating file + console handlers. No structured JSON logging, no trace IDs, no OpenTelemetry integration. Hard to correlate requests across services in production.

16.9 Heavy reliance on `psycpg2.RealDictCursor`

Every query returns `dict`-style rows. This is convenient but slower than typed tuples. For hot paths (e.g., `PendingOrderWorker` polling `pending_orders` every second), this could become a bottleneck.

16.10 Schema migrations are manual `DO $$ ... END $$` blocks

No Alembic, no migration framework. Schema changes require editing `init.sql` and adding `IF NOT EXISTS` checks. Forward-only; no down-migrations. Acceptable for a self-hosted project but doesn't scale to a SaaS with frequent schema changes.

16.11 No retry / DLQ for failed orders

`pending_orders` has `max_attempts` (default 10) but no dead-letter queue. Failed orders just sit with `status='failed'` and `last_error` text. No automated retry policy beyond the worker's 10-attempt loop.

16.12 Legacy SQLite references

Some code paths still use `?` placeholders (SQLite-style), which `db_postgres.py` auto-converts to `%s`. This is a smell — indicates the codebase was migrated from SQLite and not fully cleaned up.

16.13 No multi-currency PnL

All PnL is computed in the quote currency of the symbol (typically USDT for crypto, USD for stocks). No FX conversion layer for cross-currency portfolio aggregation.

16.14 Limited asset class coverage for live trading

Live trading supports crypto (10 exchanges), US stocks (IBKR), and Forex (MT5). No live trading for CN stocks, HK stocks, MOEX, or futures — those markets are data-only.

17. Reuse Candidates for VIXOR (Top 10)

Ranked by (value to VIXOR ÷ effort to port). All are Apache-2.0 licensed (code), so reuse is permitted as long as QuantDinger branding is removed for derivative distributions (per TRADEMARKS.md §5).

17.1 ★ Safe code execution sandbox (`app/utils/safe_exec.py`)

- **Why:** 470 LOC, self-contained, no internal dependencies beyond `app.utils.logger`. Three-layer defense (regex + AST + restricted builtins) with cross-platform timeout and optional subprocess isolation. Battle-tested against user-submitted Python strategy code.
- **Effort:** Low — copy `safe_exec.py`, swap the logger import, write tests for the security boundary (which QuantDinger lacks).
- **VIXOR use:** Execute user-submitted strategy / indicator code with strong isolation. Critical for any platform that lets users write Python strategies.

17.2 ★ Agent Gateway auth + jobs + audit pattern (`app/utils/agent_auth.py` + `app/utils/agent_jobs.py` + `app/routes/agent_v1/`)

- **Why:** Clean, well-documented implementation of capability-class scoped tokens (R/W/B/N/C/T), per-token rate limiting, Idempotency-Key with DB unique partial index, async job runner with SSE progress streaming, redacted audit logging. The SaaS-mode issuance-time guard is particularly well-designed.
- **Effort:** Medium — needs adaptation to VIXOR's auth/DB layer, but the core pattern (token hash at rest, scope enum, in-process rate limiter, job table with progress JSONB) is portable.
- **VIXOR use:** Give external AI agents (Cursor, Claude Code, custom automations) scoped, auditable access to VIXOR's research/backtest/ trading surface.

17.3 ★ Backtest engine (`app/services/backtest.py` — core ~2000 LOC of the 4974)

- **Why:** Mature backtest engine with multi-timeframe precision (1m/5m for crypto), long/short/both trade directions, leverage, commission/ slippage, fixed + trailing SL/TP, liquidation detection, candle-path simulation, equity-curve + trade-ledger persistence. Engine version stamped (`strategy-backtest-v1`).
- **Effort:** High — 5K LOC, tightly coupled to QuantDinger's data sources and DB schema. Better to extract the simulation core (`_simulate_signals_with_mtf`, `_infer_candle_path`, trade state machine) and re-wrap with VIXOR's data layer.
- **VIXOR use:** Drop-in backtest engine for any indicator-driven strategy. The `run_strategy_snapshot()` entrypoint is already snapshot-based and framework-agnostic.

17.4 ★ Strategy runtime + IndicatorStrategy DSL (`app/services/strategy_script_runtime.py` + `app/services/indicator_params.py`)

- **Why:** The `IndicatorStrategy` DSL (Python + `df['buy']/df['sell']`
 - `output = {...} + @strategy/@param` annotations) is a clean, low-friction authoring model. The `ScriptStrategy` event-driven model (`on_init/on_bar` + `ctx.buy/sell/close_position`) is a clean escape hatch for stateful strategies. Both share storage and use the same `safe_exec` sandbox.
- **Effort:** Medium — needs the `safe_exec` sandbox (§17.1) and a backtest engine (§17.3) to be useful. The DSL itself is ~600 LOC across the two files.
- **VIXOR use:** Let users write strategies in Python without learning a proprietary DSL. The `@param` annotation system gives the IDE auto-generated parameter sliders for free.

17.5 ★ Experiment orchestration (`app/services/experiment/`)

- **Why:** 5 files, ~700 LOC total. Rule-based regime detection (5 regimes, 6 features), structured grid/random parameter tuning, LLM- driven multi-round optimization with early-stop. Clean separation of concerns (regime / evolution / scoring / runner / prompts).
- **Effort:** Medium — depends on a backtest engine (§17.3) and LLM service (§17.7). The regime detector and scoring service are standalone.
- **VIXOR use:** Automate the "AI researches → backtests → scores → proposes best strategy" loop. The scoring service's multi-factor weighted score (return / sharpe / drawdown / stability / winrate / *profitfactor*) with letter grades is a reusable primitive.

17.6 ★ Live trading adapter pattern

([app/services/live_trading/](#))

- **Why:** 11 crypto exchanges + IBKR + MT5, all behind a uniform `BaseRestClient` interface with [ping/get_account/place_order/cancel_order/get_positions/get_fee_rate](#). Factory pattern with demo/testnet awareness. Per-exchange symbol normalization. `LiveOrderResult` dataclass standardizes results.
- **Effort:** High — each exchange client is 200–800 LOC of REST signing logic. Pick the exchanges VIXOR needs (probably Binance, OKX, Bybit, Coinbase to start) and port those.
- **VIXOR use:** Skip building exchange integrations from scratch. The direct-REST approach (no ccxt for orders) gives finer control over signing, hedging, broker codes, and demo/testnet routing than ccxt allows.

17.7 ★ Multi-provider LLM service ([app/services/llm.py](#))

- **Why:** 629 LOC, 7 providers (OpenRouter, OpenAI, Gemini, DeepSeek, Grok, MiniMax, Custom OpenAI-compatible). Auto-detect by API key. Provider-specific error messages. Model name normalization (strips `vendor/` prefix when calling direct providers). Fallback model support.
- **Effort:** Low — copy [llm.py](#), swap the config imports. Depends on `requests` only.
- **VIXOR use:** Avoid vendor lock-in to a single LLM provider. Let users bring their own key. The auto-detect-by-key pattern means zero config for users who already have an OpenAI/DeepSeek/Gemini key.

17.8 ★ Multi-channel signal notifier

([app/services/signal_notifier.py](#))

- **Why:** 912 LOC, 6 channels (browser, email, Telegram, SMS via Twilio, Discord webhook, generic HMAC-signed webhook). User-timezone-aware timestamps. Per-strategy `notification_config` JSON schema. HTML-formatted messages.
- **Effort:** Low-Medium — copy [signal_notifier.py](#), swap DB imports. Each channel is ~100 LOC and independent.
- **VIXOR use:** Out-of-the-box multi-channel notifications for strategy signals, portfolio alerts, AI analysis triggers.

17.9 ★ Data source resilience layer

([app/data_sources/circuit_breaker.py](#) + [rate_limiter.py](#) + [cache_manager.py](#))

- **Why:** ~600 LOC total. `CircuitBreaker` (CLOSED/OPEN/HALF_OPEN state machine), `RateLimiter` (min-interval + jitter), `retry_with_backoff` decorator (exponential + jitter), `DataCache` (thread-safe LRU + TTL). Pre-instantiated limiters for common CN financial APIs.

- **Effort:** Low — copy the three files. No internal dependencies.
- **VIXOR use:** Survive flaky public financial APIs. The circuit breaker pattern is essential for any platform that aggregates from multiple data sources.

17.10 ★ Credential encryption (`app/utils/credential_crypto.py`)

- **Why:** 50 LOC, self-contained. Fernet encryption with key derived from `SECRET_KEY` via `SHA-256` → `urlsafe-b64`. Encrypt/decrypt functions for JSON blobs. Wrong-key raises `ValueError` (fail-closed).

- **Effort:** Trivial — copy the file, set `SECRET_KEY`.
- **VIXOR use:** Encrypt exchange API keys / broker credentials at rest.

The pattern (derive Fernet key from a single server secret) is simple, auditable, and well-understood.

Honorable mentions (not in top 10 but worth considering)

- `SafeJSONProvider` (`app/__init__.py`) — 30 LOC, sanitizes NaN/Inf to `null` in all JSON responses. Trivial to port.
- `MarketRegimeService` (`app/services/experiment/regime.py`) — standalone regime classifier with 5 regimes and feature extraction. Useful as a strategy-selection router.
- `StrategyScoringService` (`app/services/experiment/scoring.py`) — multi-factor scoring with letter grades. Reusable for any strategy ranking UI.
- **Self-calibration loop** (`app/services/reflection.py` + `app/services/ai_calibration.py`) — practical pattern for making AI decisions self-tuning. Requires a `qd_analysis_memory`-style table.
- **OAuth2 service** (`app/services/oauth_service.py`) — 715 LOC, cross-worker-safe state, Google + GitHub. Useful if VIXOR needs social login.
- **USDT-TRC20 payment service** (`app/services/usdt_payment_service.py`) — HD-derived per-order addresses, TronGrid on-chain reconciliation. Useful if VIXOR accepts crypto payments.
- **Cross-sectional strategy guide** (`docs/CROSS_SECTIONAL_STRATEGY_GUIDE_EN.md`) — multi-symbol portfolio strategies with rebalancing. The `trading_config.cs_strategy_type = 'cross_sectional'` schema is documented but the execution logic is in `trading_executor.py` and harder to extract cleanly.

Appendix A: File-size quick reference

File	LOC	Notes
app/services/backtest.py	4,974	The backtest engine — largest file in the codebase
app/services/trading_executor.py	3,849	Live trading executor
app/services/fast_analysis.py	2,805	AI fast analysis
app/services/pending_order_worker.py	2,439	Order dispatch worker
app/services/market_data_collector.py	2,217	AI data collector
app/data_sources/polymarket.py	1,225	Polymarket data source
app/services/portfolio_monitor.py	1,770	Portfolio monitor
app/services/strategy.py	1,374	Strategy CRUD
app/services/usdt_payment_service.py	830	USDT payment
app/services/mt5_trading/client.py	858	MT5 client
app/services/oauth_service.py	715	OAuth service
app/services/billing_service.py	758	Billing
app/services/strategy_compiler.py	689	Strategy codegen
app/services/llm.py	629	LLM service
app/services/ibkr_trading/client.py	555	IBKR client
app/utils/safe_exec.py	470	Safe exec sandbox
app/utils/agent_jobs.py	339	Agent job runner
app/utils/agent_auth.py	470	Agent auth
migrations/init.sql	1,117	DB schema
app/services/experiment/runner.py	609	Experiment orchestrator
app/services/ai_calibration.py	342	AI calibration

File	LOC	Notes
app/services/reflection.py	101	Reflection worker
app/services/experiment/registry.py	170	Regime detector
app/services/experiment/scoring.py	140	Strategy scorer
app/services/experiment/evolution.py	123	Variant generator
app/utils/credential_crypto.py	50	Credential encryption

Appendix B: Environment variables (key set)

Category	Variables
Required	SECRET_KEY, ADMIN_USER, ADMIN_PASSWORD, DATABASE_URL
Deployment mode	QUANTDINGER_DEPLOYMENT_MODE (self/saas/hosted/shared/multitenant), ALLOW_LOCAL_DESKTOP_BROKERS
Agent Gateway	AGENT_LIVE_TRADING_ENABLED, AGENT_JOBS_MAX_WORKERS
LLM	LLM_PROVIDER, OPENROUTER_API_KEY, OPENAI_API_KEY, GOOGLE_API_KEY, DEEPSEEK_API_KEY, GROK_API_KEY, MINIMAX_API_KEY, CUSTOM_API_KEY, CUSTOM_API_URL, CUSTOM_MODEL, AI_CODE_GEN_MODEL
Data	FINNHUB_API_KEY, TWELVE_DATA_API_KEY, TIINGO_API_KEY, ADANOS_API_KEY, COINGLASS_API_KEY, CRYPTOQUANT_API_KEY, TAVILY_API_KEYS, SERPAPI_KEYS
Crypto trading	CCXT_DEFAULT_EXCHANGE, CCXT_TIMEOUT, PROXY_URL, LIVE_TRADING_SSL_VERIFY, LIVE_TRADING_CA_BUNDLE
Notifications	TELEGRAM_BOT_TOKEN, SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD, SMTP_FROM, SMTP_USE_TLS, SMTP_USE_SSL, TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_FROM_NUMBER, SIGNAL_NOTIFY_TIMEOUT_SEC
OAuth	OAUTH_GOOGLE_CLIENT_ID, OAUTH_GOOGLE_CLIENT_SECRET, OAUTH_GITHUB_CLIENT_ID, OAUTH_GITHUB_CLIENT_SECRET, OAUTH_STATE_TTL_MINUTES, FRONTEND_URL

Category	Variables
Payment	USDT_PAY_ENABLED, USDT_TRC20_XPUB, TRONGRID_BASE_URL, TRONGRID_API_KEY, USDT_TRC20_CONTRACT, USDT_PAY_CONFIRM_SECONDS, USDT_PAY_EXPIRE_MINUTES
Workers	ENABLE_PORTFOLIO_MONITOR, ENABLE_PENDING_ORDER_WORKER, ENABLE_REFLECTION_WORKER, REFLECTION_WORKER_INTERVAL_SEC, ENABLE_OFFLINE_AI_CALIBRATION, AI_CALIBRATION_MARKETS, AI_CALIBRATION_LOOKBACK_DAYS, AI_CALIBRATION_MIN_SAMPLES, DISABLE_RESTORE_RUNNING_STRATEGIES, STRATEGY_MAX_THREADS, PENDING_ORDER_STALE_SEC, POSITION_SYNC_ENABLED, POSITION_SYNC_INTERVAL_SEC
DB pool	DB_POOL_MIN, DB_POOL_MAX, DB_POOL_ACQUIRE_TIMEOUT, DB_POOL_HEALTH_CHECK, PG_MAX_CONNECTIONS, PG_SHARED_BUFFERS
Gunicorn	GUNICORN_WORKERS, GUNICORN_THREADS, GUNICORN_LOG_LEVEL, PYTHON_API_HOST, PYTHON_API_PORT, PYTHON_API_DEBUG
Cache	CACHE_ENABLED, CACHE_EXPIRE, REDIS_HOST, REDIS_PORT, REDIS_PASSWORD, REDIS_DB
Calibration	AI_CALIBRATION_CANDIDATE_ABS_THRESHOLDS
Reflection	REFLECTION_MIN_AGE_DAYS, REFLECTION_VALIDATE_LIMIT
Pricing	PRICE_CACHE_TTL_SEC

Appendix C: Reproduction commands

```
# Clone and inspect
cd /home/z/my-project/audit/QuantDinger

# Local dev (backend only)
cd backend_api_python
cp env.example .env
# Edit .env: set SECRET_KEY, ADMIN_USER, ADMIN_PASSWORD, DATABASE_URL
pip install -r requirements.txt
python run.py # starts Flask dev server on :5000

# Full Docker stack
cd /home/z/my-project/audit/QuantDinger
cp backend_api_python/env.example backend_api_python/.env
docker compose up -d --build
# Frontend: http://localhost:8888
# Backend: http://localhost:5000
```

```
# Postgres: 127.0.0.1:5432
# Redis:    127.0.0.1:6379

# Tests
cd backend_api_python && pytest tests/ -v

# MCP server (standalone)
cd mcp_server && pip install -e .
QUANTDINGER_BASE_URL=http://localhost:8888 \
QUANTDINGER_AGENT_TOKEN=qd_agent_xxx \
quantdinger-mcp
```

End of inventory. Total source files inspected: ~120. Total LOC read in detail: ~30,000+ across 40+ files. Report generated for VIXOR build-vs-reuse decision.